Índice

1. Introducción 1
   - 1.1. ¿Por qué construir un emulador de procesador desde cero? . . . 1
   - 1.2. Qué gana un alumno al saber implementar un emulador de un procesador real . . 1
   - 1.3. Razón de implementar un emulador para el procesador MIPS . . . 2

2. Desarrollo de la actividad 2
   - 2.1. Descripción del emulador . . . 2
     - 2.1.1. Procesador seleccionado y ALU . . . 3
     - 2.1.2. Ciclo de la CPU e instrucciones soportadas . . . 4
   - 2.2. Evidencia de ejecución del código . . . 4
     - 2.2.1. Capturas de ejecución . . . 6

3. Uso de inteligencias artificiales para la generación de código 13
   - 3.1. Prompt 1 — Yadid Soto (análisis del sistema de almacenamiento) . . . 13
   - 3.2. Prompt 2 — José de Jesús Berman (plan de pruebas unitarias) . . . 18
   - 3.3. Prompt 3 — José de Jesús Berman (plan de pruebas de integración) . . . 21
   - 3.4. Prompt 4 — José de Jesús Berman (plan de pruebas de sistema) . . . 26
   - 3.5. Prompt 5 — Miguel Damián (código de factorial; prompt literal de la práctica) . 30
   - 3.6. Prompt 6 — Yadid Soto (sumatoria) . . . 32
   - 3.7. Prompt 7 — Miguel Damián (integral definida) . . . 34
   - 3.8. Prompt 8 — Yadid Soto (promedio de un vector) . . . 36
   - 3.9. Prompt 9 — Miguel Damián (perceptrón) . . . 39
   - 3.10. Prompt 10 — Yadid Soto (clase ALU) . . . 41
   - 3.11. Prompt 11 — Miguel Damián (ventana gráfica del emulador) . . . 44
   - 3.12. Prompt 12 — José de Jesús Berman (documentación del conjunto de instrucciones) 47
   - 3.13. Prompt 13 — Equipo (reestructuración final a cinco paquetes) . . . 52
   - 3.14. Reflexión sobre el uso de la inteligencia artificial . . . 57

4. Reflexiones finales 57
   - 4.1. Nivel de programación que se requiere . . . 57
   - 4.2. El trabajo en equipo en el diseño e implementación de un procesador . . . 57
   - 4.3. Por qué les conviene a quienes cursarán esta experiencia el próximo año . . . 58
   - 4.4. Valores en juego . . . 58

5. Referencias 59

1. Introducción

Los procesadores que dan vida a los dispositivos que usamos a diario se perciben como una caja negra: se sabe que ejecutan instrucciones, pero rara vez se tiene la oportunidad de construir uno para observar qué ocurre realmente dentro. En esta práctica desarrollamos, desde cero, un emulador del procesador MIPS en Java, con la intención de sustituir la memorización de conceptos por la experiencia de implementarlos.

1.1 ¿Por qué construir un emulador de procesador desde cero?

Un procesador comercial real no puede estudiarse pieza por pieza: contiene millones de transistores, su diseño es propiedad industrial y se construye en equipos grandes a lo largo de años. Diseñar el nuestro desde cero nos permitió tomar y ver cada una de las decisiones que en un chip comercial vienen ya tomadas: el ancho de palabra, el mapa de direcciones, el conjunto de instrucciones, el manejo de los desbordamientos y la forma en que se actualiza el contador de programa.

Además, implementar el ciclo de instrucción (búsqueda, decodificación, ejecución, memoria y escritura) desmitifica el hardware: cada instrucción que escribe un programador en un lenguaje de alto nivel termina, al final, en operaciones de la ALU y en movimientos de datos entre registros y memoria. Y algo importante desde el punto de vista didáctico: en un procesador real un error cuesta silicio, mientras que en un emulador un error es una prueba fallida que se puede corregir, repetir y documentar.

1.2 Qué gana un alumno al saber implementar un emulador de un procesador real

Quien ha construido un emulador deja de ver la arquitectura de computadoras como una asignatura de memorización y la entiende como un sistema que se puede razonar: sabe qué registros intervienen en una suma, qué significa que un dato desborde y por qué un bucle consume más ciclos que otro. En lo práctico, adquiere programación de bajo nivel (manejo de bits, registros, memoria y aritmética con y sin signo), capacidad de leer documentación técnica y traducirla a código, y disciplina de ingeniería de software: diseño modular, pruebas unitarias, pruebas de integración y pruebas de sistema.

A esto se suma una ventaja profesional cada vez más valorada: la capacidad de usar la inteligencia artificial generativa con criterio, es decir, de pedirle apoyo a la herramienta y de comprobar después que lo que ésta devuelve es correcto.

1.3 Razón de implementar un emulador para el procesador MIPS

Elegimos MIPS de entre las opciones disponibles (RISC-V, x86, M1 y AMD) por varias razones. Primero, es una arquitectura RISC con sólo tres formatos de instrucción fijos (R, I y J), de modo que la codificación de 32 bits es regular y predecible, lo que la hace factible de implementar en un curso sin sacrificar rigor. Segundo, es el procesador de referencia en la literatura de arquitectura de computadoras (por ejemplo, en los libros de Patterson y Hennessy), por lo que existe abundante material para resolver dudas. Tercero, su flujo de control es transparente: el PC se incrementa en una palabra por cada instrucción y las ramas se calculan con desplazamientos relativos, lo que permite observar con claridad cómo se modifica el flujo de un programa. Por último, sus servicios de sistema (`syscall`) permiten resolver la entrada y salida de datos sin necesidad de emular periféricos complejos.

2. Desarrollo de la actividad

2.1 Descripción del emulador

El emulador se implementó en Java con una arquitectura modular de cinco paquetes grandes, siguiendo el esqueleto del diagrama de clases que un compañero del equipo compartió como referencia:

Estructura del proyecto

```
src/
|-- cpu/           # CPU (ciclo de 5 etapas IF/ID/EX/MEM/WB) y PC
|-- ejecucion/     # EjecutorInstrucciones, UnidadControl,
    SenalesControl
|-- memoria/       # MemoriaSistema, UnidadAlmacenamiento, ROM, RAM,
|                  # DiscoDuro y Registros
|-- alu/           # ALU y enum OperacionALU (incluye LUI)
|-- instruccion/   # Instruccion, Decodificador, Opcode, Funct,
|                  # FormatoInstruccion, ServicioSyscall,
    ProgramaBuilder,
|                  # Desensamblador, Binario
|-- ui/            # Emulador (texto) y VentanaEmulador (Swing)
|-- programas/     # Sumatoria, Factorial, Integral, Promedio,
    Perceptron
`-- pruebas/       # MiniTest, PruebasUnitarias, PruebasIntegracion,
                   # PruebasSistema, EjecutorPruebas
```

Listing 1. Estructura de carpetas del repositorio

La memoria está dividida en segmentos con dirección: la ROM guarda las instrucciones en las direcciones 0 a 99 y se bloquea una vez cargado el programa; la RAM guarda los datos del programa en las direcciones 100 a 199; y el DiscoDuro conserva información persistente en las direcciones 200 a 299. Todas las unidades trabajan con una palabra de 32 bits y 100 celdas.

El procesador cuenta con 32 registros de 32 bits, donde `$zero` siempre lee cero e ignora las escrituras, y con un contador de programa (PC) de 32 bits que contiene la dirección de la siguiente instrucción a ejecutar: durante la búsqueda de instrucciones el PC se incrementa para apuntar a la siguiente, y las ramas (`beq`, `bne`, `j`, `jal`, `jr`) escriben en él la dirección destino.

Además, la CPU nunca muere: arranca en la dirección 0, el PC es circular dentro del mapa de memoria, la celda 0 contiene un NOP y existe un límite de seguridad de 100 000 ciclos; sólo se detiene con la instrucción propia `halt` (opcode `0x3F`) o con el servicio `syscall` 10.

2.1.1 Procesador seleccionado y ALU

Se seleccionó el procesador MIPS [x] frente a RISC-V, x86, M1 y AMD. La ALU procesa 32 bits y soporta trece operaciones: ADD, ADDU, SUB, SUBU, AND, OR, XOR, NOR, SLT, SLTU, SLL, SRL y LUI, además de NOP. ADD y SUB detectan desbordamiento con signo y

detienen la emulación; ADDU y SUBU envuelven el resultado sin avisar. MIPS no tiene registros de banderas, por lo que las comparaciones se hacen con `slt/slti/sltu` seguidas de `beq` o `bne`.

2.1.2 Ciclo de la CPU e instrucciones soportadas

Cada instrucción pasa por cinco etapas:

> IF (instrucción = ROM[PC]; PC = PC + 1) → ID (separar opcode, rs, rt, rd, shamt, funct o inmediato y generar las señales de control) → EX (la ALU calcula) → MEM (`lw` lee la memoria, `sw` la escribe) → WB (el resultado llega al registro destino).

El conjunto soportado incluye las instrucciones de formato R (`add`, `addu`, `sub`, `subu`, `and`, `or`, `xor`, `nor`, `slt`, `sltu`, `sll`, `srl`, `mult`, `div`, `mflo`, `mfhi`, `jr`, `syscall`), de formato I (`addi`, `addiu`, `andi`, `ori`, `xori`, `slti`, `sltiu`, `lui`, `lw`, `sw`, `beq`, `bne`) y de formato J (`j`, `jal`), con cinco servicios de `syscall`: imprimir entero (1), imprimir cadena (4), leer entero (5), imprimir carácter (11) y terminar (10). Los programas de usuario se arman con `ProgramaBuilder`, que emite las instrucciones y resuelve las etiquetas de las ramas, sin pasar por un archivo de texto ensamblado.

2.2 Evidencia de ejecución del código

Las pruebas técnicas se ejecutaron con la orden `java -cp out pruebas.EjecutorPruebas` y arrojaron 117 pruebas unitarias, 44 pruebas de integración y 40 pruebas de sistema, todas pasadas y ninguna fallida. La primera tabla reúne la ejecución de los cinco programas de usuario y de las suites de pruebas; la segunda muestra la traza de la ejecución del programa factorial línea por línea, con el resultado que arrojó el emulador. De cada ejecución se capturó el estado al inicio y al final, que se muestran en las figuras siguientes.

Tabla 1. Ejecución de los programas de usuario (Paso 4) y de las suites de pruebas

Línea de código a ejecutar	Resultado del emulador
`java -cp out ui.Emulador sumatoria`	“Sumatoria de 3 a 10 = 52” (68 ciclos)
`java -cp out ui.Emulador factorial`	“Factorial de 7 = 5040” (63 ciclos)
`java -cp out ui.Emulador integral`	“Integral de x^2 dx desde x=1 hasta x=5 = 30” (56 ciclos)
`java -cp out ui.Emulador promedio`	“Vector: 4 8 15 16 23” y “Promedio = 13” (123 ciclos)
`java -cp out ui.Emulador perceptron`	“z = 3 y = 1” (62 ciclos)

Línea de código a ejecutar	Resultado del emulador
`java -cp out ui.Emulador factorial --traza --estado`	Traza por ciclo con el PC y el código en hexadecimal; volcado de registros y mapa de memoria
`java -cp out pruebas.PruebasUnitarias`	RESUMEN PRUEBAS UNITARIAS / Pasadas: 117 Fallidas: 0
`java -cp out pruebas.PruebasIntegracion`	RESUMEN PRUEBAS DE INTEGRACION / Pasadas: 44 Fallidas: 0
`java -cp out pruebas.PruebasSistema`	RESUMEN PRUEBAS DE SISTEMA / Pasadas: 40 Fallidas: 0
`java -cp out pruebas.EjecutorPruebas`	201 pruebas en total: resultado global, todas pasaron

Tabla 2. Traza de la ejecución del programa factorial, línea por línea

Línea de código a ejecutar	Resultado del emulador
`lw $t0, 100($zero)` (PC = 0)	$t0 = Mem[100] = 7
`addiu $v0, $zero, 4` (PC = 1)	$v0 = 4 (servicio: imprimir cadena)
`addiu $a0, $zero, 101` (PC = 2)	$a0 = 101 (dirección de la cadena “Factorial de ”)
`syscall` (PC = 3)	Salida en pantalla: “Factorial de ”
`addu $a0, $t0, $zero` (PC = 4)	$a0 = 7
`addiu $v0, $zero, 1` (PC = 5)	$v0 = 1 (servicio: imprimir entero)
`syscall` (PC = 6)	Salida en pantalla: “7”
`addiu $v0, $zero, 4` (PC = 7)	$v0 = 4 (servicio: imprimir cadena)
`addiu $a0, $zero, 115` (PC = 8)	$a0 = 115 (dirección de la cadena “ =”)
`syscall` (PC = 9)	Salida en pantalla: “ = ”
`addiu $t1, $zero, 1` (PC = 10)	$t1 = 1 (acumulador del factorial)
`addiu $t2, $zero, 1` (PC = 11)	$t2 = 1 (contador del bucle)
`slt $t3, $t0, $t2` (PC = 12, 1.ª vez)	$t3 = 0, porque $7 < 1$ es falso
`bne $t3, $zero, salida` (PC = 13, 1.ª vez)	No salta ($t3 = 0); PC = 14
`mult $t0, $t2` (PC = 14, 1.ª vez)	LO:HI = $7 \times 1 = 7$

Línea de código a ejecutar	Resultado del emulador
`mflo $t1` (PC = 15, 1.ª vez)	$t1 = LO = 7
`addiu $t2, $t2, 1` (PC = 16, 1.ª vez)	$t2 = 2
`j 12` (PC = 17)	PC = 12 (regresa al inicio del bucle)
Bucle PC 12 a 17 repetido hasta completar 7 multiplicaciones	$t1 = 7 × 6 × 5 × 4 × 3 × 2 × 1 = 5040; $t2 = 8
`slt $t3, $t0, $t2` (PC = 12, 8.ª vez)	$t3 = 1, porque 7 < 8 es verdadero
`bne $t3, $zero, salida` (PC = 13, 8.ª vez)	Sí salta: PC = 18 (sale del bucle)
`addu $a0, $t1, $zero` (PC = 18)	$a0 = 5040
`addiu $v0, $zero, 1` (PC = 19)	$v0 = 1 (servicio: imprimir entero)
`syscall` (PC = 20)	Salida en pantalla: “5040”
`addiu $v0, $zero, 4` (PC = 21)	$v0 = 4 (servicio: imprimir cadena)
`addiu $a0, $zero, 119` (PC = 22)	$a0 = 119 (dirección del salto de línea)
`syscall` (PC = 23)	Salida en pantalla: salto de línea
`addiu $v0, $zero, 10` (PC = 24)	$v0 = 10 (servicio: terminar el programa)
`halt` (PC = 25)	La CPU se detiene con HALT; 63 ciclos; salida final: “Factorial de 7 = 5040”

2.2.1 Capturas de ejecución

Se muestra, para cada programa, el estado al inicio y al fin de la ejecución, tal como lo reporta el emulador.

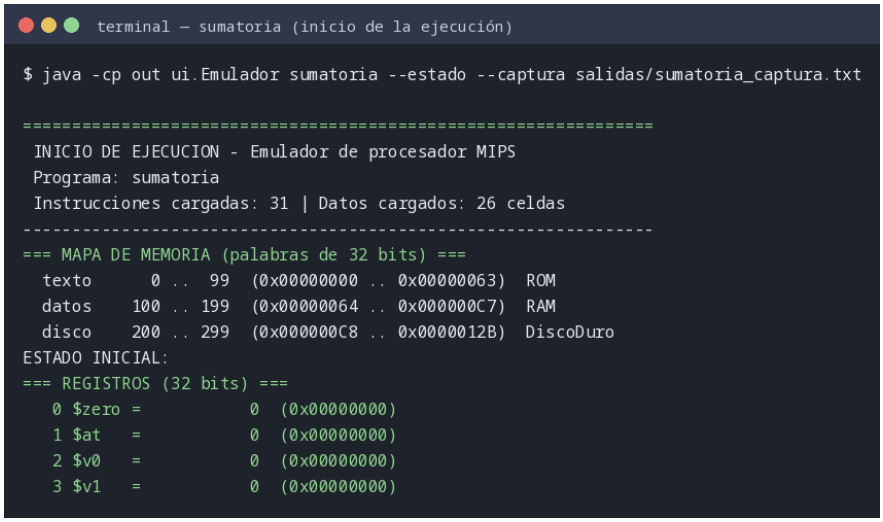


Figura 1. Sumatoria: estado inicial (todos los registros en 0 y el mapa de memoria segmentado).

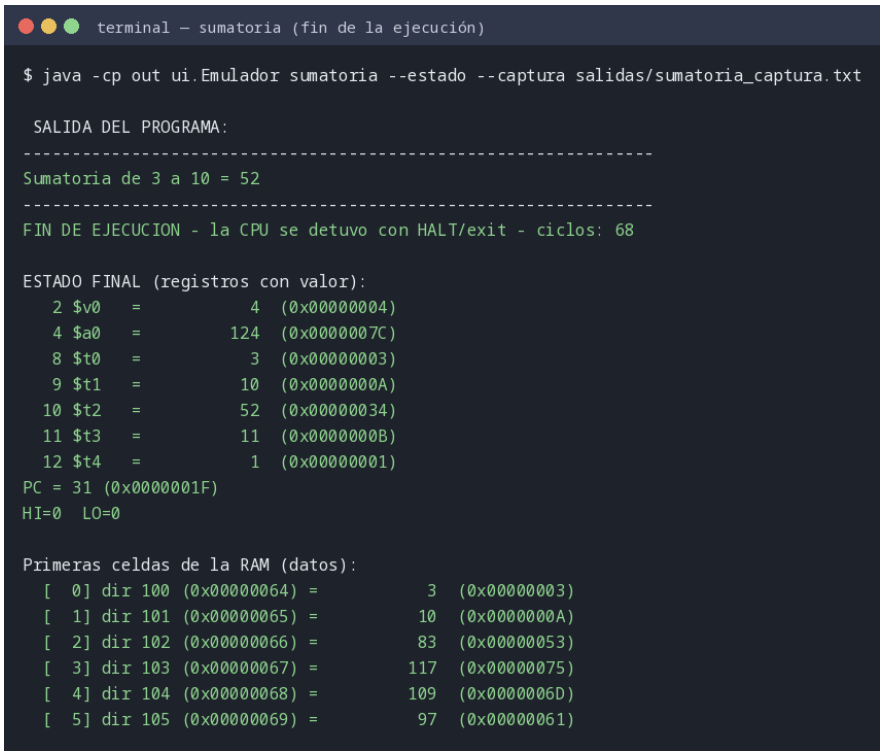


Figura 2. Sumatoria: fin de la ejecución; resultado 52 en 68 ciclos.

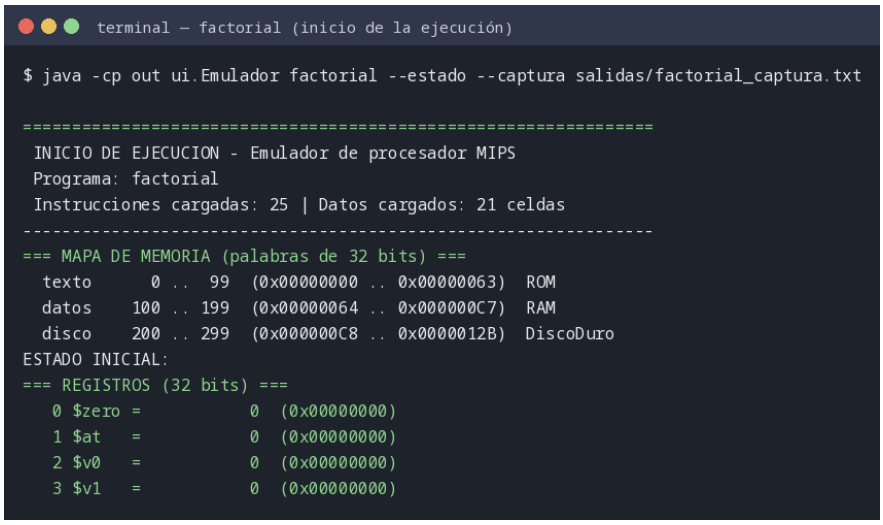


Figura 3. Factorial: estado inicial de la emulación.

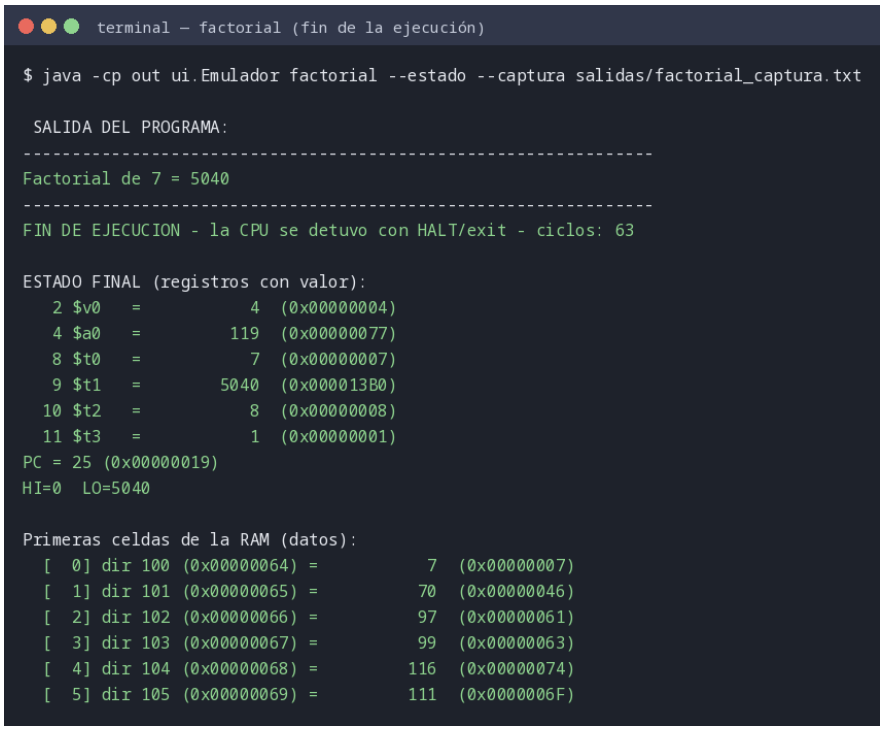


Figura 4. Factorial: fin de la ejecución; $t1 = 5040 y LO = 5040 en 63 ciclos.

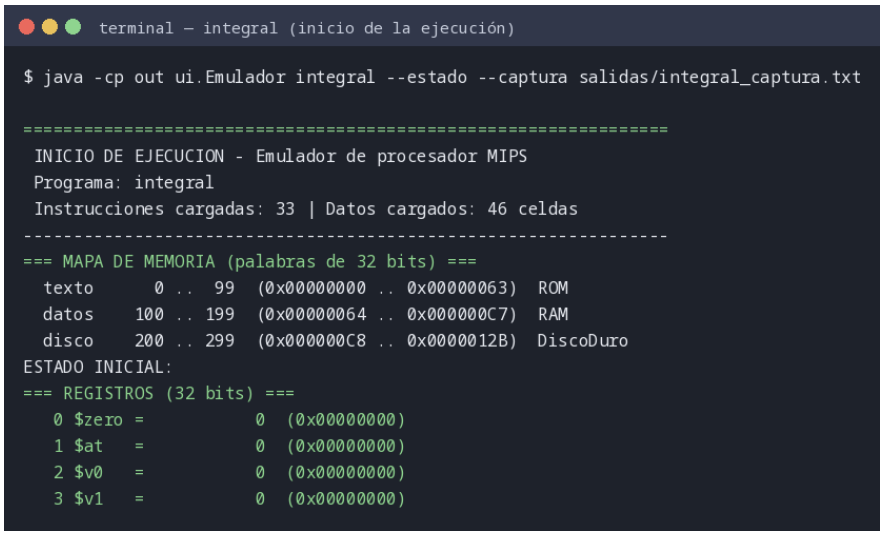


Figura 5. Integral: estado inicial de la emulación.

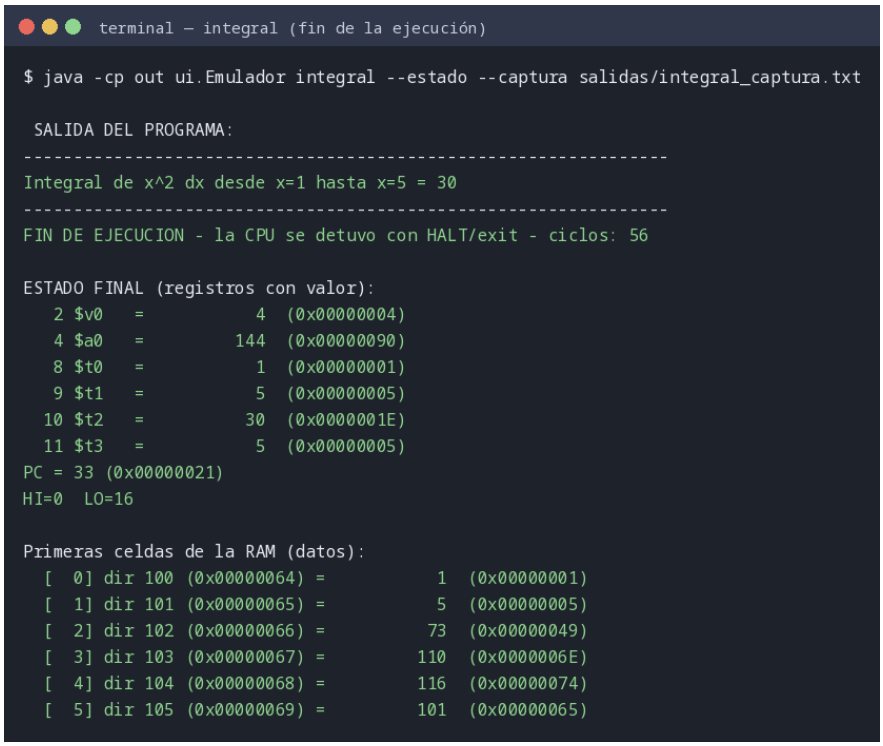


Figura 6. Integral: fin de la ejecución; resultado 30 en 56 ciclos.

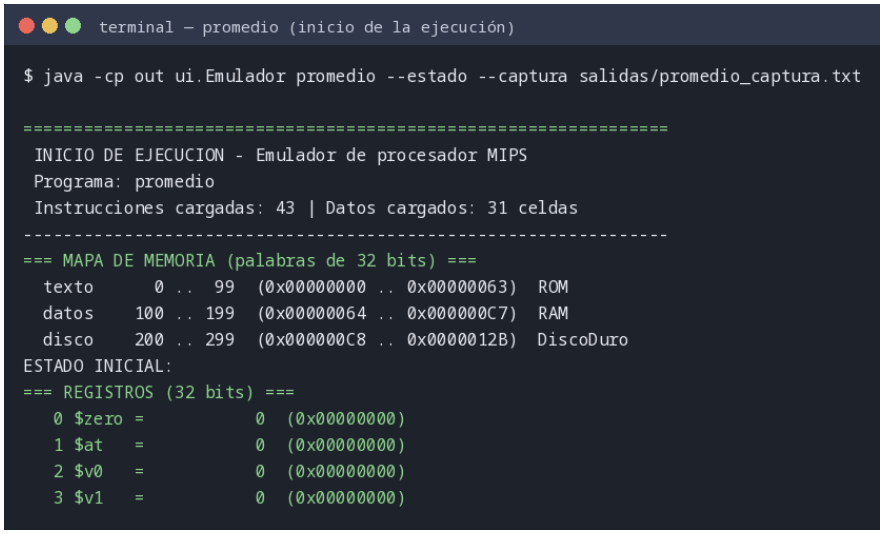


Figura 7. Promedio: estado inicial de la emulación.

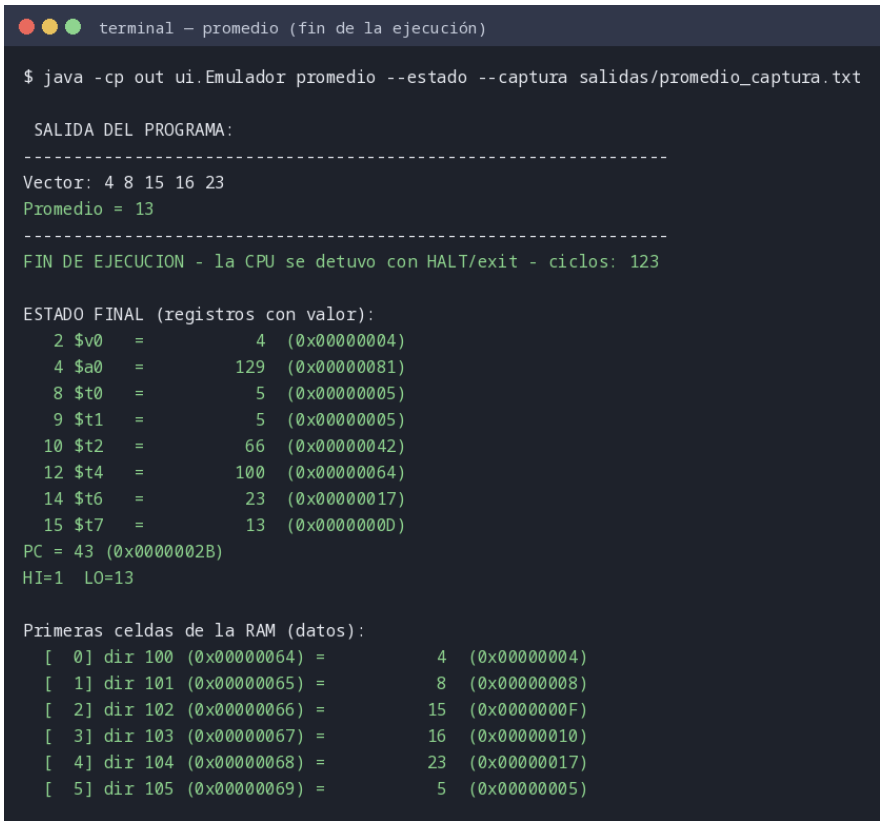


Figura 8. Promedio: fin de la ejecución; promedio 13 en 123 ciclos.

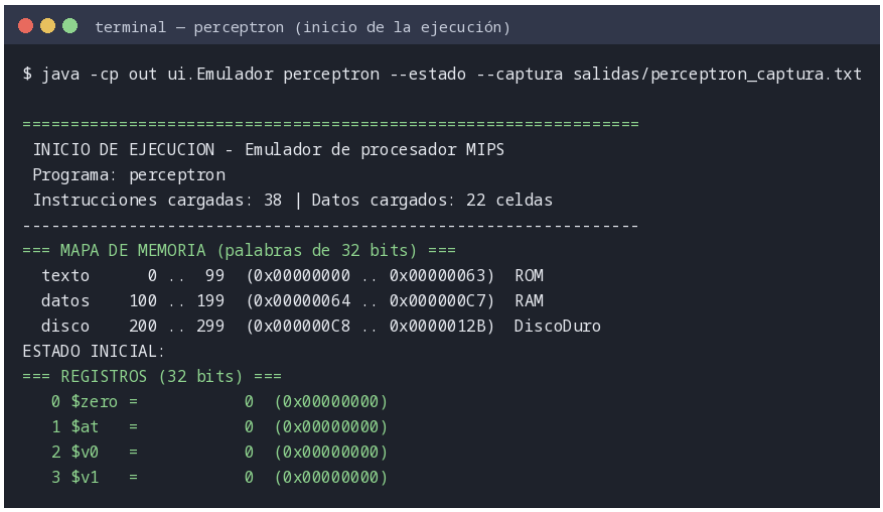


Figura 9. Perceptrón: estado inicial de la emulación.

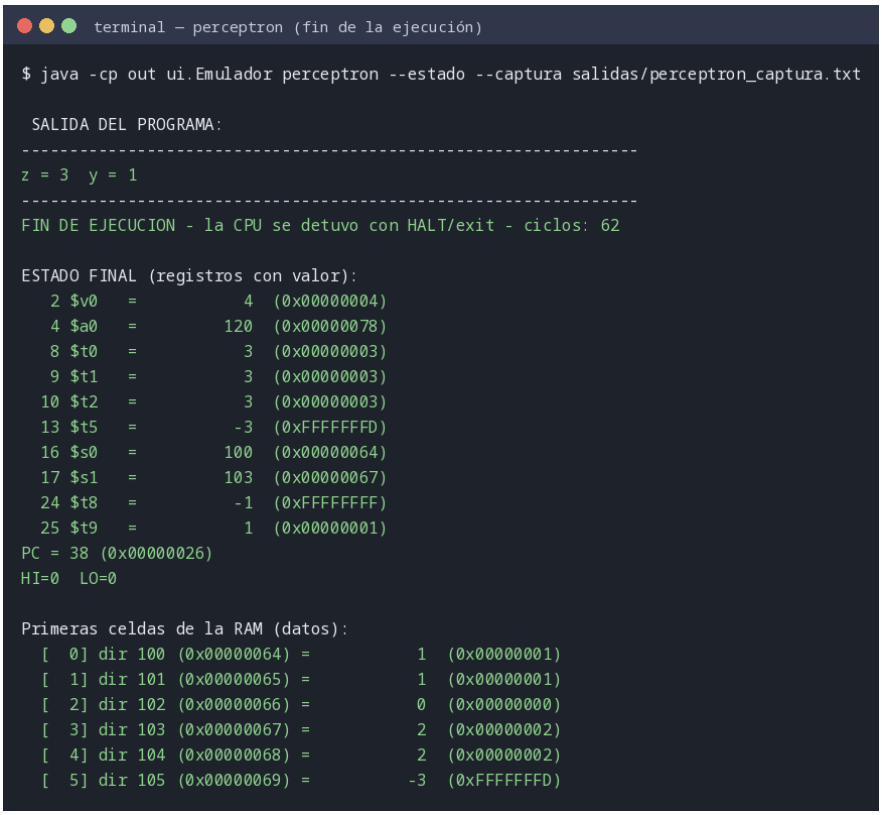


Figura 10. Perceptrón: fin de la ejecución; z = 3, y = 1 en 62 ciclos.

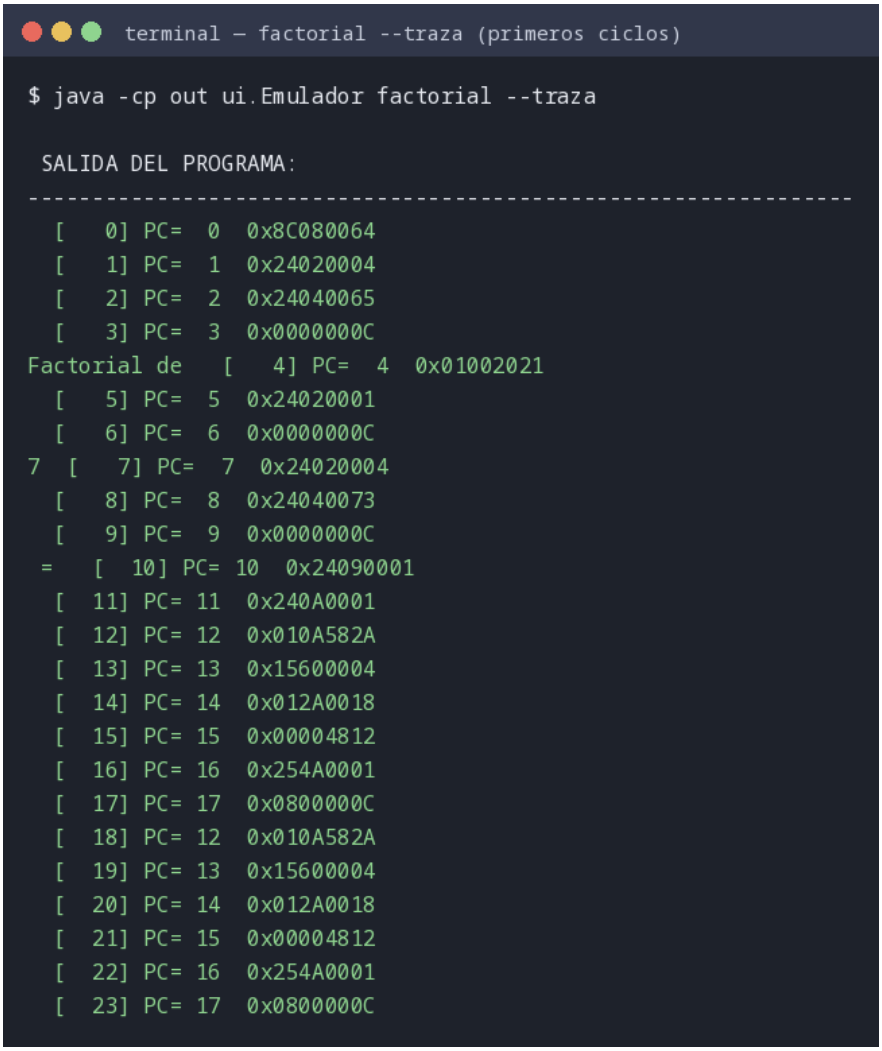


Figura 11. Factorial con `--traza`: primeros ciclos con el PC y el código de la instrucción en hexadecimal.

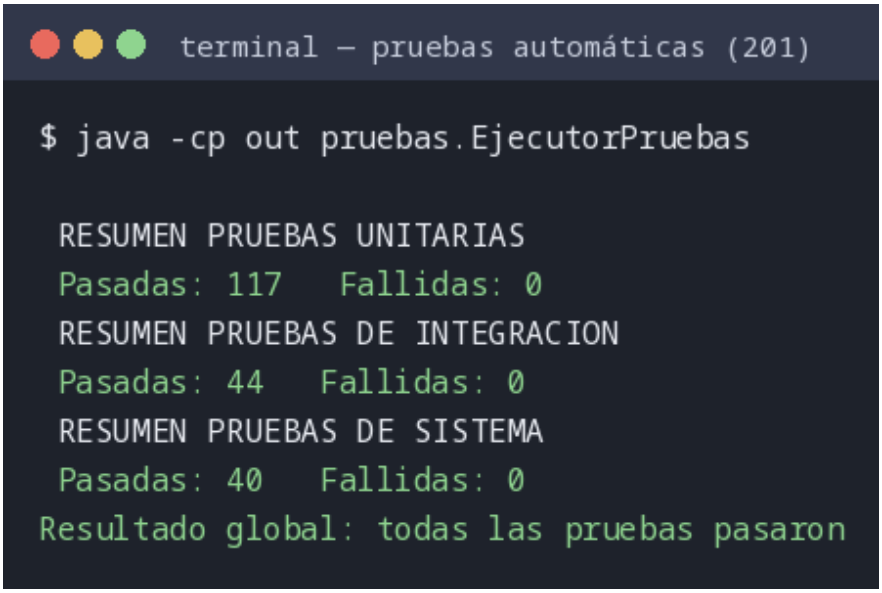


Figura 12. Ejecución de las 201 pruebas automáticas: 117 unitarias, 44 de integración y 40 de sistema, todas pasadas.

3. Uso de inteligencias artificiales para la generación de código

La inteligencia artificial generativa se utilizó como herramienta de apoyo en tres frentes: el análisis del diseño, la generación de los planes de prueba y la redacción de los programas en ensamblador MIPS. Los prompts fueron escritos por el equipo y ajustados de forma colaborativa; la respuesta de la herramienta nunca se usó tal cual: se ejecutó contra las suites de pruebas y se corrigió hasta que el comportamiento fue el esperado.

A continuación se presentan, en orden, los trece prompts utilizados, quién los formuló y qué parte del proyecto produjo cada uno, junto con la captura de pantalla de la conversación.

3.1 Prompt 1 — Yadid Soto (análisis del sistema de almacenamiento)

Prompt

«Analiza las siguientes clases de mi proyecto en Java: `UnidadAlmacenamiento`, `RAM`, `ROM`, `Registros` y contador de programa. `<CÓDIGO>` ¿Qué unidades funcionales faltan para emular un procesador completo y cómo las reorganizarías en paquetes sin romper el código existente?»

Resultado: arquitectura modular en cinco paquetes: `cpu`, `ejecucion`, `memoria`, `alu` y `instruccion`; la clase `MemoriaSistema` con el mapa de direcciones segmentado y el documento `docs/arquitectura.md`.

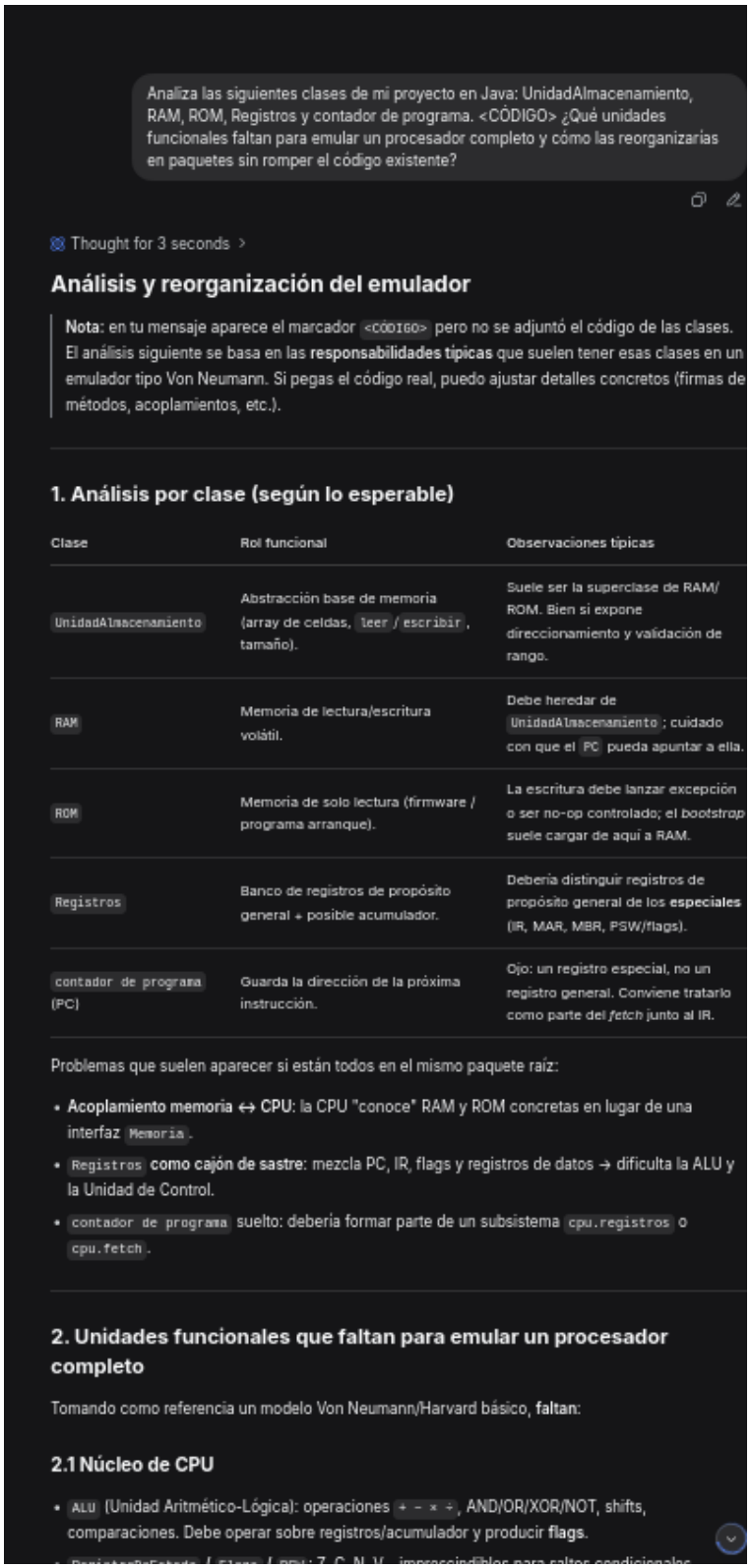
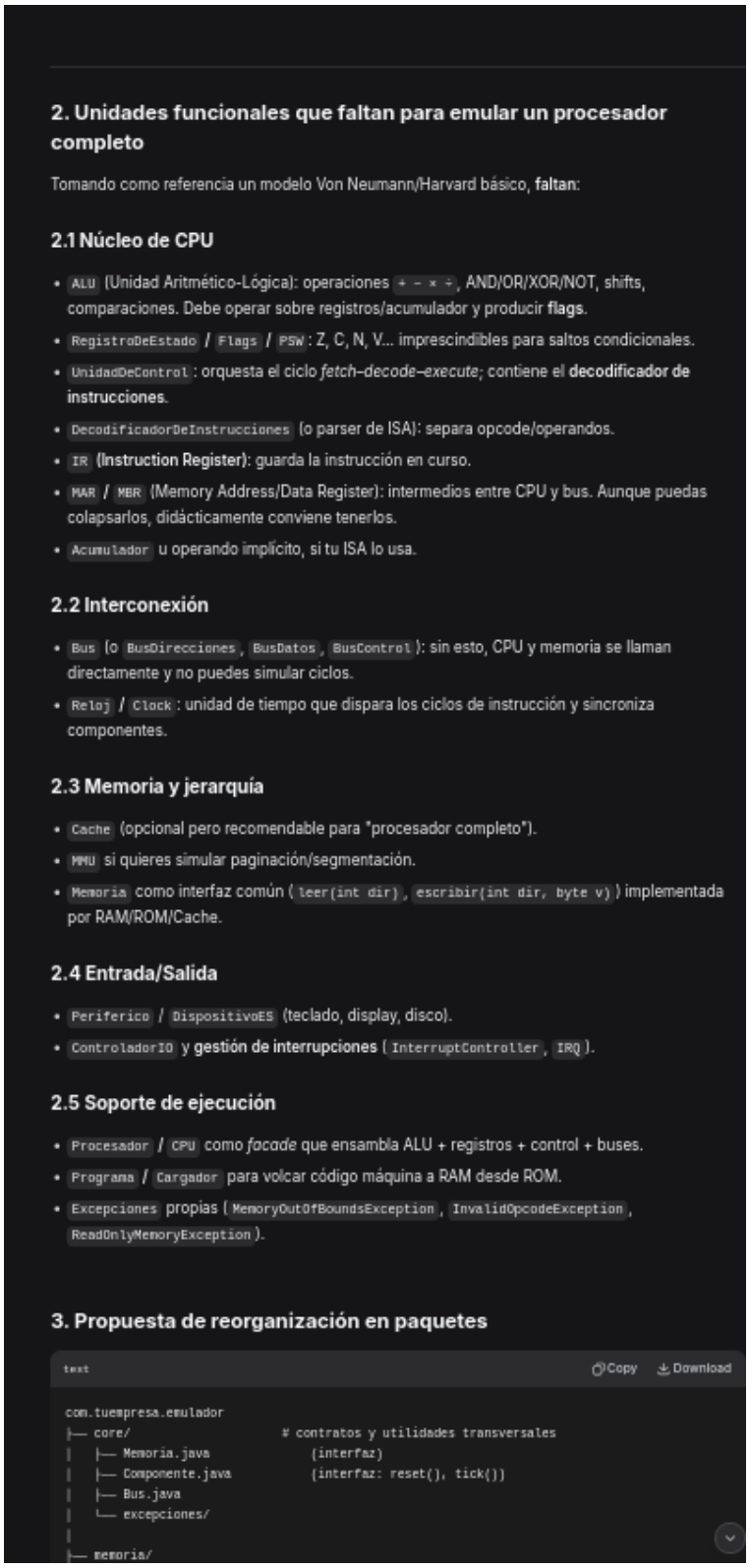


Figura 13. Prompt 1: solicitud de análisis de las clases de almacenamiento y de qué unidades faltan.



2. Unidades funcionales que faltan para emular un procesador completo

Tomando como referencia un modelo Von Neumann/Harvard básico, **faltan**:

2.1 Núcleo de CPU

- `ALU` (Unidad Aritmético-Lógica): operaciones `+ - × ÷`, AND/OR/XOR/NOT, shifts, comparaciones. Debe operar sobre registros/acumulador y producir **flags**.
- `RegistroDeEstado` / `Flags` / `PSW`: Z, C, N, V... imprescindibles para saltos condicionales.
- `UnidadDeControl`: orquesta el ciclo *fetch–decode–execute*; contiene el **decodificador de instrucciones**.
- `DecodificadorDeInstrucciones` (o parser de ISA): separa opcode/operandos.
- `IR` **(Instruction Register)**: guarda la instrucción en curso.
- `MAR` / `MBR` (Memory Address/Data Register): intermedios entre CPU y bus. Aunque puedas colapsarlos, didácticamente conviene tenerlos.
- `Acumulador` u operando implícito, si tu ISA lo usa.

2.2 Interconexión

- `Bus` (o `BusDirecciones`, `BusDatos`, `BusControl`): sin esto, CPU y memoria se llaman directamente y no puedes simular ciclos.
- `Reloj` / `Clock`: unidad de tiempo que dispara los ciclos de instrucción y sincroniza componentes.

2.3 Memoria y jerarquía

- `Cache` (opcional pero recomendable para "procesador completo").
- `MMU` si quieres simular paginación/segmentación.
- `Memoria` como interfaz común (`leer(int dir)`, `escribir(int dir, byte v)`) implementada por RAM/ROM/Cache.

2.4 Entrada/Salida

- `Periferico` / `DispositivoES` (teclado, display, disco).
- `ControladorIO` y **gestión de interrupciones** (`InterruptController`, `IRQ`).

2.5 Soporte de ejecución

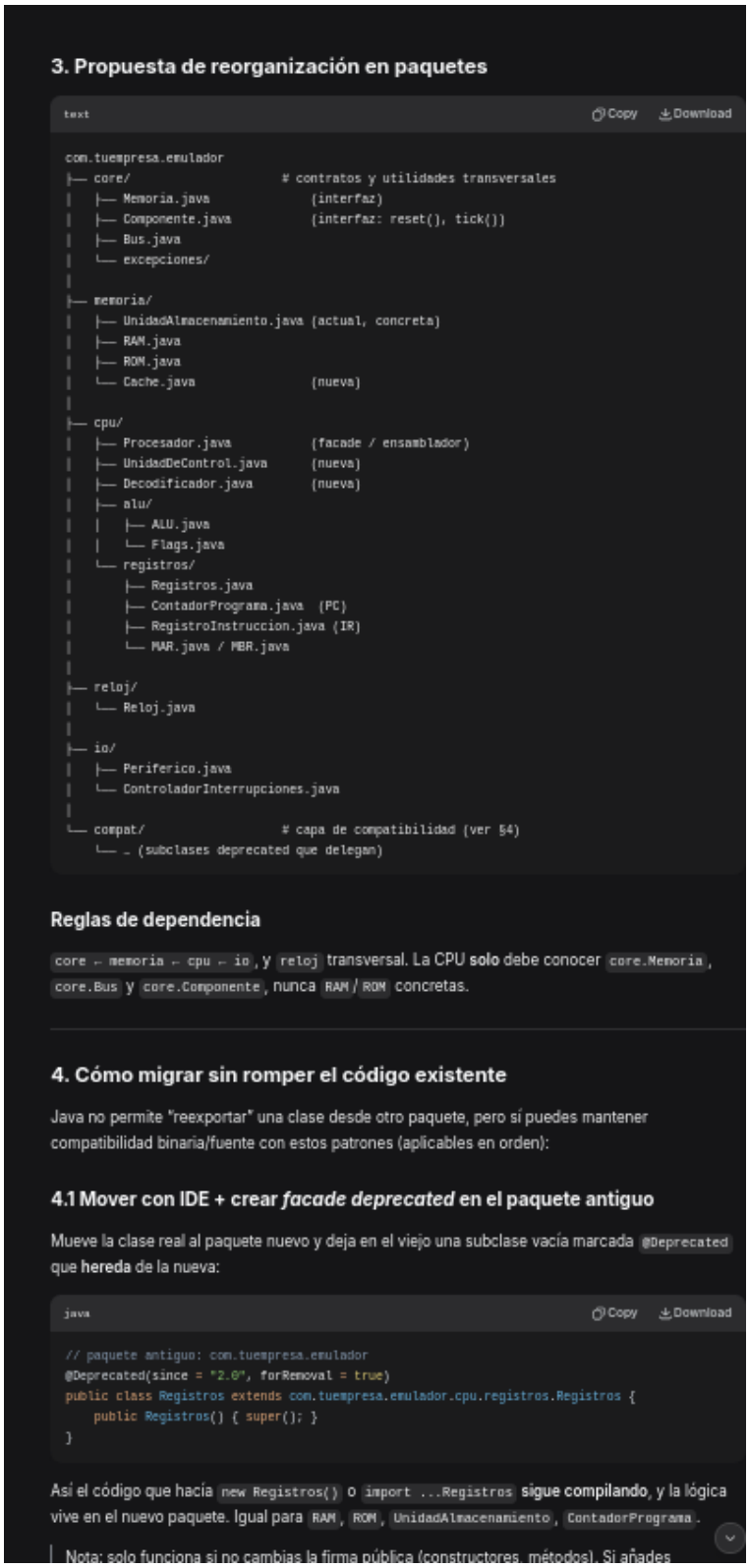
- `Procesador` / `CPU` como *facade* que ensambla ALU + registros + control + buses.
- `Programa` / `Cargador` para volcar código máquina a RAM desde ROM.
- `Excepciones` propias (`MemoryOutOfBoundsException`, `InvalidOpcodeException`, `ReadOnlyMemoryException`).

3. Propuesta de reorganización en paquetes

text Copy Download

```
com.tuempresa.emulador
├── core/                    # contratos y utilidades transversales
│   ├── Memoria.java             (interfaz)
│   ├── Componente.java          (interfaz: reset(), tick())
│   ├── Bus.java
│   └── excepciones/
│
├── memoria/
```

Figura 14. Prompt 1, captura 2 de 4: unidades funcionales que faltan (núcleo de CPU, interconexión, memoria y E/S).



3. Propuesta de reorganización en paquetes

text — Copy — Download

```
com.tuempresa.emulador
├── core/                    # contratos y utilidades transversales
│   ├── Memoria.java             (interfaz)
│   ├── Componente.java          (interfaz: reset(), tick())
│   ├── Bus.java
│   └── excepciones/
│
├── memoria/
│   ├── UnidadAlmacenamiento.java (actual, concreta)
│   ├── RAM.java
│   ├── ROM.java
│   └── Cache.java               (nueva)
│
├── cpu/
│   ├── Procesador.java          (facade / ensamblador)
│   ├── UnidadDeControl.java     (nueva)
│   ├── Decodificador.java       (nueva)
│   ├── alu/
│   │   ├── ALU.java
│   │   └── Flags.java
│   └── registros/
│       ├── Registros.java
│       ├── ContadorPrograma.java  (PC)
│       ├── RegistroInstruccion.java (IR)
│       └── MAR.java / MBR.java
│
├── reloj/
│   └── Reloj.java
│
├── io/
│   ├── Periferico.java
│   └── ControladorInterrupciones.java
│
└── compat/                  # capa de compatibilidad (ver §4)
    └── … (subclases deprecated que delegan)
```

Reglas de dependencia

`core` ← `memoria` ← `cpu` ← `io`, y `reloj` transversal. La CPU **solo** debe conocer `core.Memoria`, `core.Bus` y `core.Componente`, nunca `RAM`/`ROM` concretas.

---

4. Cómo migrar sin romper el código existente

Java no permite "reexportar" una clase desde otro paquete, pero sí puedes mantener compatibilidad binaria/fuente con estos patrones (aplicables en orden):

4.1 Mover con IDE + crear *facade deprecated* en el paquete antiguo

Mueve la clase real al paquete nuevo y deja en el viejo una subclase vacía marcada `@Deprecated` que **hereda** de la nueva:

java — Copy — Download

```java
// paquete antiguo: com.tuempresa.emulador
@Deprecated(since = "2.0", forRemoval = true)
public class Registros extends com.tuempresa.emulador.cpu.registros.Registros {
    public Registros() { super(); }
}
```

Así el código que hacía `new Registros()` o `import ...Registros` **sigue compilando**, y la lógica vive en el nuevo paquete. Igual para `RAM`, `ROM`, `UnidadAlmacenamiento`, `ContadorPrograma`.

> Nota: solo funciona si no cambias la firma pública (constructores, métodos). Si añades

Figura 15. Prompt 1, captura 3 de 4: árbol de paquetes propuesto y cómo migrar sin romper el código.



java Copy Download

4.3 Inyección de dependencias en lugar de `new`

Cambia en la CPU `new RAM(...)` por un campo `private final Memoria memoria;` recibido por constructor o setter. Mantén un constructor "legacy" que internamente haga `this(new RAM(...))` para no romper instanciaciones existentes.

4.4 Nuevas unidades como *composition*, no herencia

`Procesador` agrega `ALU`, `UnidadDeControl`, `Registros`. No modifiques `Registros` para "meter" la ALU dentro: pásala por constructor. Si ya hay métodos que esperaban `Registros`, sigue exponiéndolos.

4.5 Versionado y ciclo de vida

- Marca la capa `compat` como `@Deprecated(forRemoval = true)`.
- Añade un test de "compatibilidad de API" (p. ej. ArchUnit) que verifique que los paquetes antiguos solo contienen fachadas.
- Elimina `compat` en la siguiente *major*.

4.6 Ejemplo de ensamblaje final del procesador

java Copy Download

```java
public final class Procesador implements Componente {
    private final Bus bus;
    private final Memoria memoria;
    private final Registros registros;
    private final ContadorPrograma pc;
    private final ALU alu;
    private final UnidadDeControl control;

    public Procesador(Bus bus, Memoria memoria, ALU alu) {
        this.bus = bus;
        this.memoria = memoria;
        this.alu = alu;
        this.registros = new Registros();
        this.pc = new ContadorPrograma();
        this.control = new UnidadDeControl(this);
    }

    @Override public void reset() { pc.reset(); control.reset(); }
    @Override public void tick()  { control.ciclo(); }
}
```

---

5. Resumen ejecutivo

- **Faltan**: ALU, Unidad de Control + decodificador, IR/MAR/MBR, flags/PSW, bus(es), reloj, interfaz `Memoria`, periféricos/I/O, gestión de interrupciones, y opcionalmente caché/MMU.
- **Reorganización**: paquetes `core`, `memoria`, `cpu` (`alu`, `registros`, `control`), `reloj`, `io`, `compat`.
- **Compatibilidad**: mover + subclase `@Deprecated` que hereda, extraer interfaces (`Memoria`, `Componente`), inyectar dependencias con constructores *legacy* que deleguen, y aislar todo lo antiguo en `compat` para eliminarlo en una major futura.

Si pegas el `<CÓDIGO>` real, te devuelvo el refactor concreto clase por clase (incluyendo fachadas de compatibilidad con las firmas exactas de tus métodos).

Figura 16. Prompt 1, captura 4 de 4: inyección de dependencias, composición y resumen ejecutivo.

3.2 Prompt 2 — José de Jesús Berman (plan de pruebas unitarias)

Prompt

«Genera un plan de pruebas para el emulador del procesador MIPS. Considera la unidad de almacenamiento (rango de direcciones y limpieza), los registros ($zero de sólo lectura), la ALU (las operaciones y el desbordamiento), la decodificación de las instrucciones de formato R, I y J y el comportamiento del PC.»

Resultado: `docs/pruebas/plan_pruebas_unitarias.md` y `src/pruebas/PruebasUnitarias.java`: 117 pruebas automáticas, todas pasadas.



Figura 17. Prompt 2, captura 1 de 4: rango de direcciones y segmentación.



ID	Caso de prueba	Entrada / Condición	Resultado esperado
MEM-11	Estado inicial de la memoria no inicializada	Programa que lee una dirección del segmento `.bss` o del heap antes de escribir	Debe devolver `0` (muchos emuladores inicializan a cero) o un valor conocido/ documentado. No debe contener basura aleatoria.
MEM-12	Reset del emulador	Ejecutar un programa, luego reiniciar el emulador	Todos los registros (excepto `$zero`) y la memoria deben volver al estado inicial (PC en `0x00400000` , `$sp` en `0x7fffeffc` , memoria de datos limpia).
MEM-13	Limpieza selectiva de segmento	Escribir en `.data` , luego recargar el programa sin reiniciar el emulador	El segmento `.data` debe sobrescribirse co los valores del nuevo binario; las direccione no escritas deben quedar en el estado inici (típicamente 0).

2. Registros: `$zero` de solo lectura

ID	Caso de prueba	Entrada / Condición	Resultado esperado
REG-01	Lectura de `$zero`	`add $t0, $zero, $zero`	`$t0 = 0` .
REG-02	Escritura directa en `$zero` (formato R)	`add $zero, $t0, $t1` (con `$t0=5` , `$t1=10`)	`$zero` permanece en `0` ; la escritura se descarta.
REG-03	Escritura en `$zero` mediante `lw`	`lw $zero, 0($t0)` (dirección con valor `0xDEADBEEF`)	`$zero` sigue siendo `0` ; no se produce efecto.
REG-04	Escritura en `$zero` mediante `addi`	`addi $zero, $zero, 100`	`$zero` sigue siendo `0` .
REG-05	Uso de `$zero` como operando en bifurcación	`beq $zero, $zero, etiqueta`	La bifurcación siempre se toma (condici verdadera).

3. ALU: operaciones y desbordamiento

3.1 Operaciones aritméticas y lógicas (formato R e I)

ID	Instrucción	Entrada (ejemplo)	Resultado esperado
ALU-01	`add`	`$t0=0x00000005` , `$t1=0x0000000A` → `add $t2,$t0,$t1`	`$t2 = 0x0000000F`
ALU-02	`sub`	`$t0=0x0000000A` , `$t1=0x00000005` → `sub $t2,$t0,$t1`	`$t2 = 0x00000005`
ALU-03	`and`	`$t0=0xF0F0F0F0` , `$t1=0x0F0F0F0F` → `and $t2,$t0,$t1`	`$t2 = 0x00000000`
ALU-04	`or`	`$t0=0xF0F0F0F0` , `$t1=0x0F0F0F0F` → `or $t2,$t0,$t1`	`$t2 = 0xFFFFFFFF`
ALU-05	`xor`	`$t0=0xAAAAAAAA` , `$t1=0x55555555` → `xor $t2,$t0,$t1`	`$t2 = 0xFFFFFFFF`
ALU-06	`nor`	`$t0=0x00000000` , `$t1=0x00000000` → `nor $t2,$t0,$t1`	`$t2 = 0xFFFFFFFF` (NOR bit a bit)

Figura 18. Prompt 2, captura 2 de 4: limpieza de memoria, $zero y ALU.



3.2 Desbordamiento (overflow)

El desbordamiento en MIPS ocurre en sumas/restas con signo cuando el resultado excede el rango de 32 bits en complemento a dos.

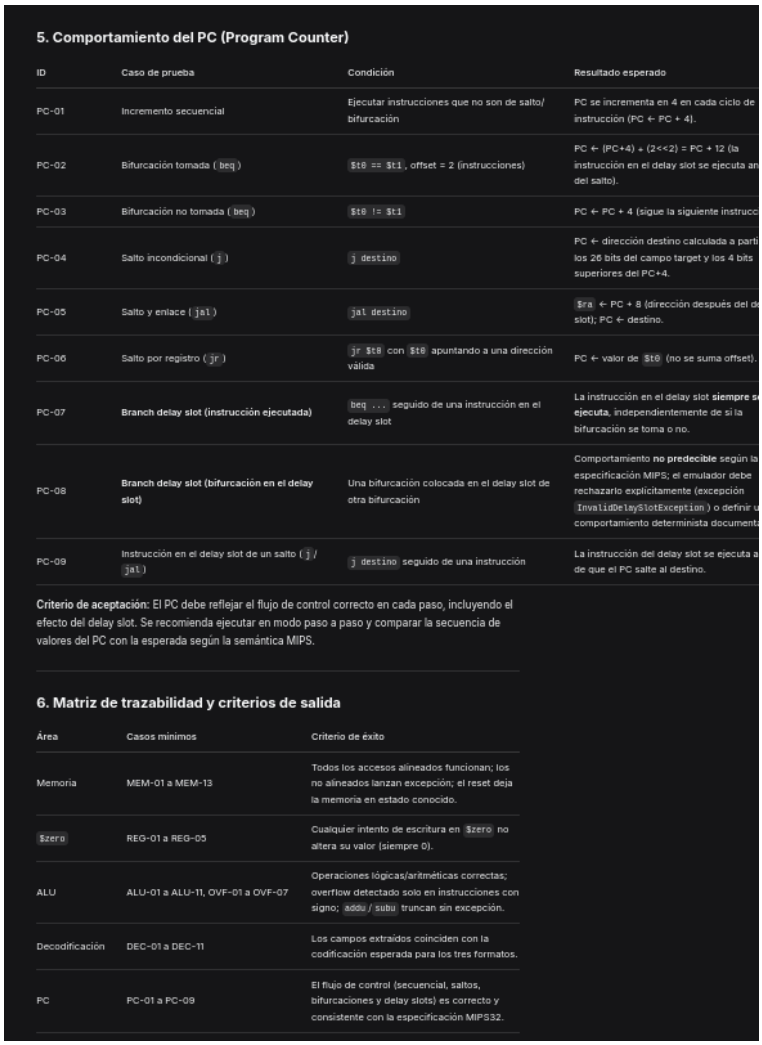
ID	Instrucción	Condición de desbordamiento	Resultado esperado
OVF-01	`add`	`$t0=0x7FFFFFFF` (máx positivo), `$t1=0x00000001` → `add $t2,$t0,$t1`	Se activa la excepción de desbordamiento (el emulador debe lanzar una excepción o registrar el flag de overflow). El registro destino no debe actualizarse con el valor truncado.
OVF-02	`add`	`$t0=0x80000000` (mín negativo), `$t1=0xFFFFFFFF` (−1) → `add $t2,$t0,$t1`	Desbordamiento (resultado negativo + negativo = positivo fuera de rango). Excepción.
OVF-03	`sub`	`$t0=0x80000000`, `$t1=0x00000001` → `sub $t2,$t0,$t1`	Desbordamiento (negativo − positivo = positivo fuera de rango). Excepción.
OVF-04	`sub`	`$t0=0x7FFFFFFF`, `$t1=0xFFFFFFFF` → `sub $t2,$t0,$t1`	Desbordamiento (positivo − negativo = negativo fuera de rango). Excepción.
OVF-05	`addi`	`$t0=0x7FFFFFFF`, inmediato `1` → `addi $t1,$t0,1`	Desbordamiento; excepción.
OVF-06	`addu` (sin overflow)	`$t0=0x7FFFFFFF`, `$t1=0x00000001` → `addu $t2,$t0,$t1`	No se lanza excepción; el resultado se trunca a `0x80000000`.
OVF-07	`subu` (sin overflow)	`$t0=0x80000000`, `$t1=0x00000001` → `subu $t2,$t0,$t1`	No hay excepción; resultado truncado `0x7FFFFFFF`.

Criterio de aceptación: Para instrucciones con overflow (`add`, `addi`, `sub`), el emulador debe activar el mecanismo de excepción correspondiente (registro `Cause` o similar) y no escribir un resultado incorrecto en el destino. Para `addu`/`subu`, el resultado debe ser el truncado a 32 bits.

4. Decodificación de instrucciones (formatos R, I, J)

ID	Formato	Instrucción de prueba	Campos esperados	Verificación
DEC-01	R	`add $t2, $t0, $t1`	`opcode=0x00`, `rs=8` ($t0), `rt = 9` ($t1), `rd=10` ($t2), `shamt=0`, `funct=0x20`	El emulador debe extraer correctament cinco campos y ejecutar la suma.
DEC-02	R	`sub $t3, $t4, $t5`	`opcode=0x00`, `rs=12`, `rt=13`, `rd=11`, `funct=0x22`	Resta correcta.
DEC-03	R	`sll $t0, $t1, 4`	`opcode=0x00`, `rs=0`, `rt=9`, `rd=8`, `shamt=4`, `funct=0x00`	Desplazamiento correcto.
DEC-04	R	`jr $ra`	`opcode=0x00`, `rs=31`, `funct=0x08`	Salto al registro `$ra`; PC se actualiza c valor de `$ra`.
DEC-05	I	`addi $t0, $t1, 100`	`opcode=0x08`, `rs=9`, `rt=8`, `imm=100`	El inmediato se extiende con signo y se suma.
DEC-06	I	`lw $t2, 16($sp)`	`opcode=0x23`, `rs=29` ($sp), `rt=10`, `imm=16`	Carga desde `$sp+16`.
DEC-07	I	`sw $t3, -4($fp)`	`opcode=0x2B`, `rs=30` ($fp), `rt=11`, `imm=-4`	Almacena en `$fp-4` (inmediato con si

Figura 19. Prompt 2, captura 3 de 4: desbordamiento y decodificación de instrucciones.



5. Comportamiento del PC (Program Counter)

ID	Caso de prueba	Condición	Resultado esperado
PC-01	Incremento secuencial	Ejecutar instrucciones que no son de salto/bifurcación	PC se incrementa en 4 en cada ciclo de instrucción (PC ← PC + 4).
PC-02	Bifurcación tomada (`beq`)	`$t0 == $t1`, offset = 2 (instrucciones)	PC ← (PC+4) + (2<<2) = PC + 12 (la instrucción en el delay slot se ejecuta an del salto).
PC-03	Bifurcación no tomada (`beq`)	`$t0 != $t1`	PC ← PC + 4 (sigue la siguiente instrucc
PC-04	Salto incondicional (`j`)	`j destino`	PC ← dirección destino calculada a parti los 26 bits del campo target y los 4 bits superiores del PC+4.
PC-05	Salto y enlace (`jal`)	`jal destino`	`$ra` ← PC + 8 (dirección después del d slot); PC ← destino.
PC-06	Salto por registro (`jr`)	`jr $t8` con `$t0` apuntando a una dirección válida	PC ← valor de `$t0` (no se suma offset).
PC-07	**Branch delay slot (instrucción ejecutada)**	`beq ...` seguido de una instrucción en el delay slot	La instrucción en el delay slot **siempre s ejecuta**, independientemente de si la bifurcación se toma o no.
PC-08	**Branch delay slot (bifurcación en el delay slot)**	Una bifurcación colocada en el delay slot de otra bifurcación	Comportamiento **no predecible** según la especificación MIPS; el emulador debe rechazarlo explícitamente (excepción `InvalidDelaySlotException`) o definir u comportamiento determinista document
PC-09	Instrucción en el delay slot de un salto (`j` / `jal`)	`j destino` seguido de una instrucción	La instrucción del delay slot se ejecuta a de que el PC salte al destino.

Criterio de aceptación: El PC debe reflejar el flujo de control correcto en cada paso, incluyendo el efecto del delay slot. Se recomienda ejecutar en modo paso a paso y comparar la secuencia de valores del PC con la esperada según la semántica MIPS.

6. Matriz de trazabilidad y criterios de salida

Área	Casos mínimos	Criterio de éxito
Memoria	MEM-01 a MEM-13	Todos los accesos alineados funcionan; los no alineados lanzan excepción; el reset deja la memoria en estado conocido.
`$zero`	REG-01 a REG-05	Cualquier intento de escritura en `$zero` no altera su valor (siempre 0).
ALU	ALU-01 a ALU-11, OVF-01 a OVF-07	Operaciones lógicas/aritméticas correctas; overflow detectado solo en instrucciones con signo; `addu` / `subu` truncan sin excepción.
Decodificación	DEC-01 a DEC-11	Los campos extraídos coinciden con la codificación esperada para los tres formatos.
PC	PC-01 a PC-09	El flujo de control (secuencial, saltos, bifurcaciones y delay slots) es correcto y consistente con la especificación MIPS32.

Figura 20. Prompt 2, captura 4 de 4: comportamiento del PC y matriz de trazabilidad.

3.3 Prompt 3 — José de Jesús Berman (plan de pruebas de integración)

Prompt

«Genera un plan de pruebas de integración de código para el emulador de procesador MIPS. Verifica el ciclo completo (IF/ID/EX/MEM/WB), la conexión `MemoriaSistema` → `CPU` → `Registros` y RAM, el funcionamiento de las ramas (beq, bne, j, jr) y el uso de los syscalls.»

Resultado: `docs/pruebas/plan_pruebas_integracion.md` y `src/pruebas/PruebasIntegracion.java`: 44 pruebas, todas pasadas.

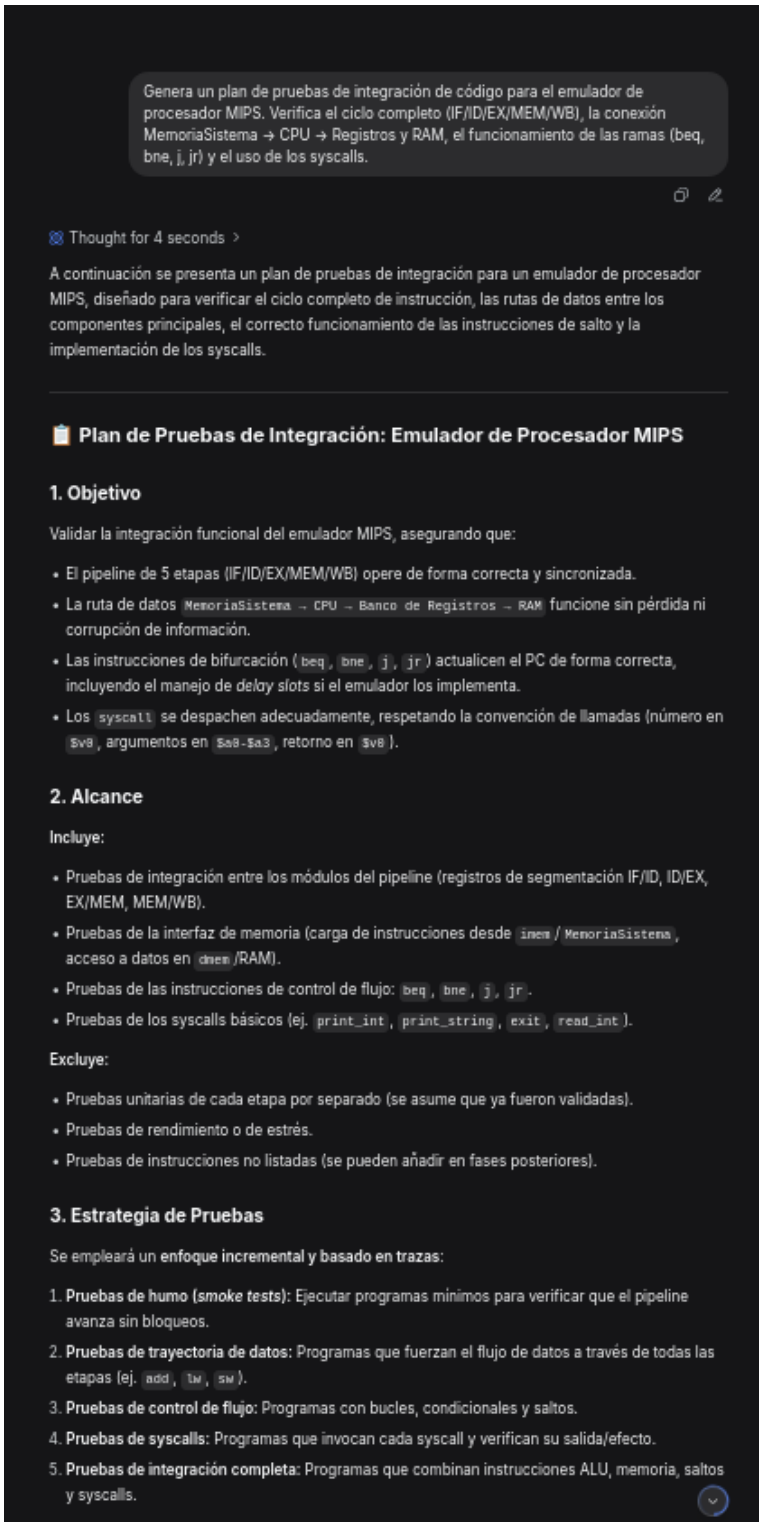


Figura 21. Prompt 3, captura 1 de 4: objetivo, alcance y estrategia de pruebas de integración.



4. Entorno de Pruebas

- **Emulador bajo prueba:** El emulador MIPS desarrollado.
- **Herramientas de referencia:** Se puede utilizar **MARS** o **SPIM** como oráculo para generar los resultados esperados en programas sencillos.
- **Formato de entrada:** Archivos de texto con el código de máquina (hexadecimal) para `imem.txt` y datos iniciales para `dmem.txt`, tal como lo usa el simulador de referencia.
- **Formato de salida:** Archivos `RFresult.txt` (estado final de registros) y `dmemresult.txt` (estado final de memoria).
- **Automatización:** Script que compile/ejecute cada caso, compare las salidas con los golden files y reporte diferencias (ignorando líneas marcadas como "don't care" si es necesario).

5. Casos de Prueba Detallados

5.1. Verificación del Ciclo Completo (IF/ID/EX/MEM/WB)

Objetivo: Asegurar que las instrucciones atraviesan las 5 etapas sin perder datos y que los registros de segmentación actualizan correctamente sus señales de control.

ID	Programa (instrucciones)	Propósito	Salida esperada

CP-01	`addi $t0, $zero, 5`
	`addi $t1, $zero, 3`
	`add $t2, $t0, $t1`
		Verifica el flujo ALU→WB y la escritura en registro.	`$t2 = 8` en `RFresult.txt`.

CP-02	`lw $t0, 0($zero)`
	`addi $t1, $t0, 1`
	`sw $t1, 4($zero)`
		Verifica la ruta Memoria→CPU→ALU→Memoria (load-use).	`$t0` con el valor de `dmem[0]`, `dmem[4] = $t0+1`.

CP-03	`addi $t0, $zero, 10`
	`sw $t0, 0($zero)`
	`lw $t1, 0($zero)`
		Verifica la coherencia entre escritura y lectura en RAM.	`$t1 = 10` en `RFresult.txt`.

CP-04	Secuencia de 5 `nop` seguidas de `addi $t0, $zero, 1`	Verifica que el pipeline avanza incluso con instrucciones que no escriben.	`$t0 = 1` tras 5 ciclos de burbuja.

Criterio de aceptación: Los valores en `RFresult.txt` y `dmemresult.txt` coinciden exactamente con los esperados. Si se habilita `STRICT_STATE=1`, la traza `stateresult.txt` debe mostrar los datos correctos en cada etapa (señales `IF/ID`, `ID/EX`, etc.).

5.2. Conexión MemoriaSistema → CPU → Registros y RAM

Objetivo: Validar que el emulador carga correctamente el programa desde la memoria de sistema (imem) y que los accesos a datos (load/store) llegan a la RAM y actualizan los registros.

ID	Programa	Propósito	Salida esperada

MC-01	`lui $t0, 0x1234`
	`ori $t0, $t0, 0x5678`
	`sw $t0, 0($zero)`
		Verifica la carga de una dirección de 32 bits desde memoria de sistema y su escritura en RAM.	`dmem[0] = 0x12345678`.

MC-02	`lw $t0, 0($zero)` (con `dmem[0]` pre-cargado con `0xCAFEBABE`)	Verifica la lectura desde RAM hacia el banco de registros.	`$t0 = 0xCAFEBABE`.
MC-03	`addi $t0, $zero, 0x100` `sw $t0, 0($t0)`	Verifica direccionamiento indirecto con registro base.	`dmem[0x100] = 0x100`.

Criterio de aceptación: Los valores en `dmemresult.txt` y `RFresult.txt` coinciden con los golden files.

5.3. Funcionamiento de las Ramas (`beq`, `bne`, `j`, `jr`)

Objetivo: Confirmar que las bifurcaciones condicionales e incondicionales actualizan el PC correctamente y que no se ejecutan instrucciones no deseadas.

Figura 22. Prompt 3, captura 2 de 4: entorno y verificación del ciclo completo y conexión memoria-CPU.



BR-01	`beq $t0, $t1, target` (con `$t0=$t1=1`)
	`addi $t2, $zero, 99`
	`target: addi $t2, $zero, 1`
		Verifica que `beq` salta cuando la condición es verdadera.	`$t2 = 1` (la instrucción `addi $t2,99` se omite).

BR-02	`bne $t0, $t1, target` (con `$t0=1, $t1=2`)
	`addi $t2, $zero, 99`
	`target: addi $t2, $zero, 2`
		Verifica que `bne` salta cuando los registros son distintos.	`$t2 = 2`.

BR-03	`j target`
	`addi $t2, $zero, 99`
	`target: addi $t2, $zero, 3`
		Verifica el salto incondicional `j`.	`$t2 = 3`.

BR-04	`jal subrutina`
	`addi $t2, $zero, 4`
	`subrutina: jr $ra`
		Verifica `jr` para retornar de una subrutina (enlace vía `$ra`).	`$t2 = 4` (el flujo regresa correctamente).

BR-05	Bucle: `addi $t0, $zero, 3`
	`loop: addi $t0, $t0, -1`
	`bne $t0, $zero, loop`
		Verifica un bucle con `bne` y actualización de registro.	Al final `$t0 = 0` (el bucle se ejecuta 3 veces).

Nota: Si el emulador implementa *delay slots*, se debe incluir un `nop` después de cada branch/jump para aislar el comportamiento del salto, o bien verificar explícitamente que la instrucción en el delay slot se ejecuta.

Criterio de aceptación: El PC final y los valores de registro coinciden con los esperados. La comparación de `RFresult.txt` debe pasar.

5.4. Uso de los Syscalls

Objetivo: Verificar que el emulador interpreta correctamente el número de syscall en `$v0`, toma los argumentos de `$a0-$a3` y produce el efecto esperado (impresión en consola, lectura de entrada, terminación del programa).

ID	Syscall	Programa	Propósito	Salida esperada

SC-01	`print_int` (v0=1)	`li $v0, 1`
		`li $a0, 42`
		`syscall`
			Verifica la impresión de un entero.	La consola muestra `42`.

SC-02	`print_string` (v0=4)	`li $v0, 4`
		`la $a0, msg`
		`syscall` (msg en `.data`)
			Verifica la impresión de una cadena.	La consola muestra la cadena `msg`.

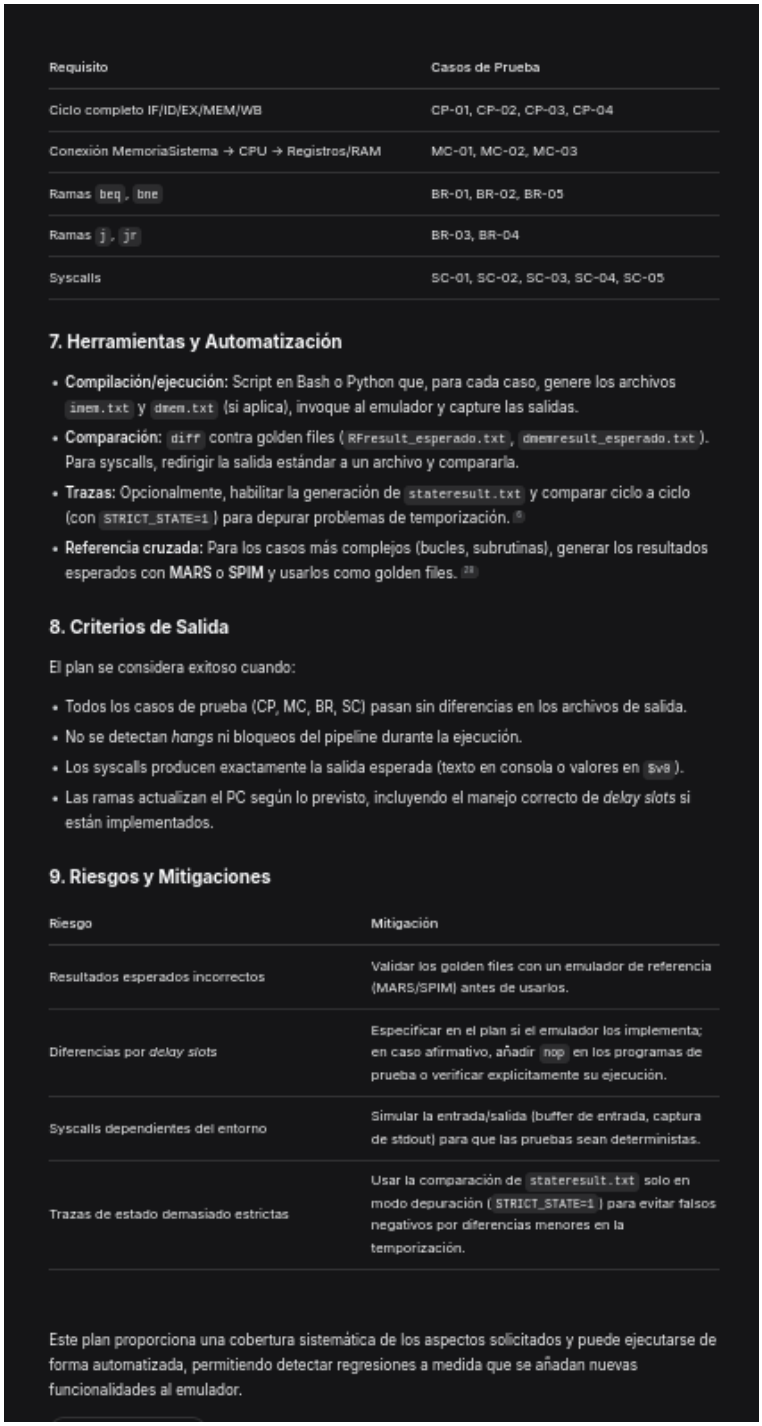
SC-03	`read_int` (v0=5)	`li $v0, 5` `syscall`	Verifica la lectura de un entero desde la entrada estándar (simulada).	`$v0` contiene el valor leído.
SC-04	`exit` (v0=10)	`li $v0, 10` `syscall`	Verifica la terminación limpia del programa.	El emulador detiene la ejecución y el PC no avanza.
SC-05	Encadenamiento de syscalls	`print_int` seguido de `print_string` y `exit`.	Verifica que el estado del procesador se mantiene entre syscalls.	Las salidas se producen en orden y el programa finaliza.

Criterio de aceptación: La salida de consola (capturada por el entorno de prueba) coincide con la esperada. Para `read_int`, se debe pre-cargar un valor en el buffer de entrada simulado y verificar que `$v0` contiene dicho valor al finalizar.

6. Matriz de Trazabilidad

Requisito	Casos de Prueba
Ciclo completo IF/ID/EX/MEM/WB	CP-01, CP-02, CP-03, CP-04
Conexión MemoriaSistema → CPU → Registros/RAM	MC-01, MC-02, MC-03
Ramas `beq`, `bne`	BR-01, BR-02, BR-05

Figura 23. Prompt 3, captura 3 de 4: funcionamiento de las ramas y uso de los syscalls.



Requisito	Casos de Prueba
Ciclo completo IF/ID/EX/MEM/WB	CP-01, CP-02, CP-03, CP-04
Conexión MemoriaSistema → CPU → Registros/RAM	MC-01, MC-02, MC-03
Ramas `beq`, `bne`	BR-01, BR-02, BR-05
Ramas `j`, `jr`	BR-03, BR-04
Syscalls	SC-01, SC-02, SC-03, SC-04, SC-05

7. Herramientas y Automatización

- **Compilación/ejecución:** Script en Bash o Python que, para cada caso, genere los archivos `imem.txt` y `dmem.txt` (si aplica), invoque al emulador y capture las salidas.
- **Comparación:** `diff` contra golden files (`RFresult_esperado.txt`, `dmemresult_esperado.txt`). Para syscalls, redirigir la salida estándar a un archivo y compararla.
- **Trazas:** Opcionalmente, habilitar la generación de `stateresult.txt` y comparar ciclo a ciclo (con `STRICT_STATE=1`) para depurar problemas de temporización.
- **Referencia cruzada:** Para los casos más complejos (bucles, subrutinas), generar los resultados esperados con **MARS** o **SPIM** y usarlos como golden files.

8. Criterios de Salida

El plan se considera exitoso cuando:

- Todos los casos de prueba (CP, MC, BR, SC) pasan sin diferencias en los archivos de salida.
- No se detectan *hangs* ni bloqueos del pipeline durante la ejecución.
- Los syscalls producen exactamente la salida esperada (texto en consola o valores en `$v0`).
- Las ramas actualizan el PC según lo previsto, incluyendo el manejo correcto de *delay slots* si están implementados.

9. Riesgos y Mitigaciones

Riesgo	Mitigación
Resultados esperados incorrectos	Validar los golden files con un emulador de referencia (MARS/SPIM) antes de usarlos.
Diferencias por *delay slots*	Especificar en el plan si el emulador los implementa; en caso afirmativo, añadir `nop` en los programas de prueba o verificar explícitamente su ejecución.
Syscalls dependientes del entorno	Simular la entrada/salida (buffer de entrada, captura de stdout) para que las pruebas sean deterministas.
Trazas de estado demasiado estrictas	Usar la comparación de `stateresult.txt` solo en modo depuración (`STRICT_STATE=1`) para evitar falsos negativos por diferencias menores en la temporización.

Este plan proporciona una cobertura sistemática de los aspectos solicitados y puede ejecutarse de forma automatizada, permitiendo detectar regresiones a medida que se añadan nuevas funcionalidades al emulador.

Figura 24. Prompt 3, captura 4 de 4: matriz de trazabilidad, herramientas y riesgos.

3.4 Prompt 4 — José de Jesús Berman (plan de pruebas de sistema)

Prompt

«Genera un plan de pruebas de sistema para el código para el emulador del procesador MIPS. Ejecuta los cinco programas de usuario (sumatoria, factorial, integral, promedio y perceptron), comprueba sus salidas exactas, que cada uno termine con `halt`/`syscall` 10 y que el número de ciclos sea razonable.»

Resultado: `docs/pruebas/plan_pruebas_sistema.md` y `src/pruebas/PruebasSistema.java`: 40 pruebas, todas pasadas.



Figura 25. Prompt 4, captura 1 de 4: objetivo, alcance y entorno de pruebas de sistema.



Caso TS-01 — Sumatoria

Campo	Valor
Programa	`sumatoria.asm`
Entrada	N = 10 (suma 1..10)
Salida esperada	`55` (exacta)
Terminación	Última instrucción ejecutada = `syscall` con `$v0 = 10`; emulador retorna exit code 0
Ciclos aceptables	150 – 600 ciclos (bucle lineal de 10 iteraciones + overhead)
Comando	`./mips-emulator sumatoria.asm --trace-cycles`

Caso TS-02 — Factorial

Campo	Valor
Programa	`factorial.asm`
Entrada	N = 5
Salida esperada	`120` (exacta)
Terminación	`syscall 10` ; exit code 0
Ciclos aceptables	120 – 500 ciclos (5 iteraciones)
Comando	`./mips-emulator factorial.asm --trace-cycles`

Caso TS-03 — Integral

Campo	Valor
Programa	`integral.asm`
Entrada	$\int_0^1 x^2\,dx$ con 100 subintervalos (regla trapezoidal)
Salida esperada	`0.333350` (±1e-4 por redondeo de punto flotante) — si usa enteros escalados, valor exacto según escala (p. ej. `3333` con factor 10^4)
Terminación	`syscall 10` ; exit code 0
Ciclos aceptables	1 500 – 8 000 ciclos (100 iteraciones con FP/multiplicaciones)
Comando	`./mips-emulator integral.asm --trace-cycles`

Caso TS-04 — Promedio

Campo	Valor
Programa	`promedio.asm`
Entrada	Arreglo `[18, 29, 30, 48, 58]`
Salida esperada	`30` (exacta; si es entero, sin decimales)

Figura 26. Prompt 4, captura 2 de 4: casos TS-01 (sumatoria) a TS-04 (promedio).



Programa	`promedio.asm`
Entrada	Arreglo `[18, 20, 30, 40, 58]`
Salida esperada	`30` (exacta; si es entero, sin decimales)
Terminación	`syscall 10` ; exit code 0
Ciclos aceptables	200 – 800 ciclos
Comando	`./mips-emulator promedio.asm --trace-cycles`

Caso TS-05 — Perceptron

Campo	Valor
Programa	`perceptron.asm`
Entrada	Compuerta AND con 4 patrones `(0,0)=0, (0,1)=0, (1,0)=0, (1,1)=1` , η=1, pesos iniciales 0, máx. épocas = 100
Salida esperada	Convergencia con pesos finales exactos (p. ej. `w0=-1, w1=1, w2=1` según convención) y número de épocas reportado (típicamente 5)
Terminación	`syscall 10` ; exit code 0
Ciclos aceptables	2 000 – 20 000 ciclos (depende de épocas hasta convergencia; debe ser < límite de épocas × ciclos/época)
Comando	`./mips-emulator perceptron.asm --trace-cycles`

6. Verificación de `halt` / `syscall 10`

Procedimiento para cada programa:

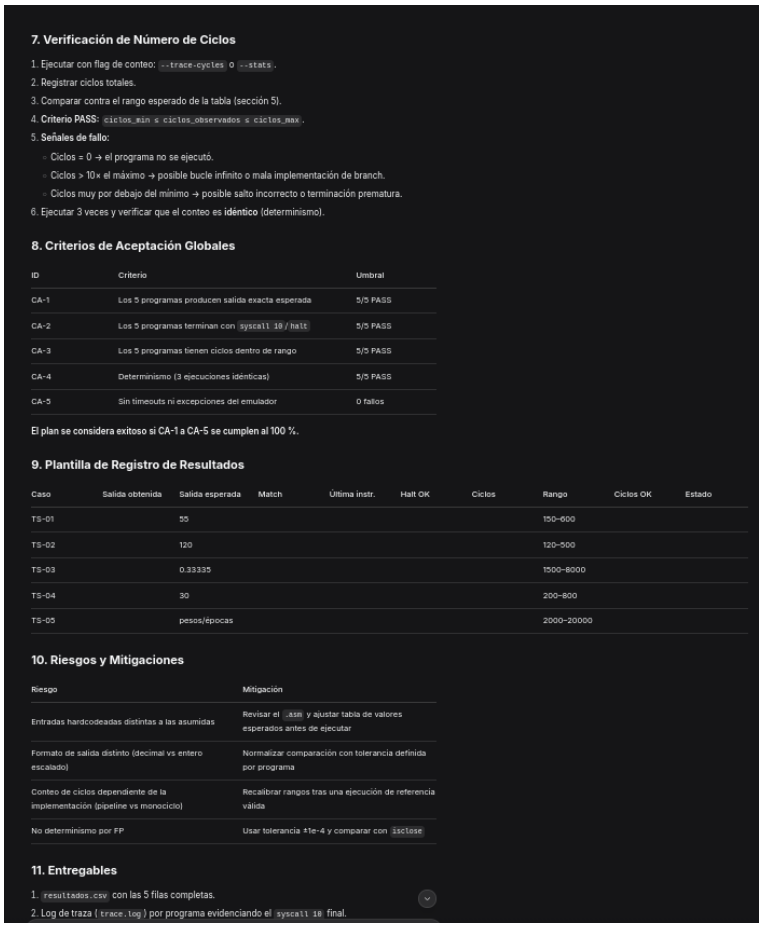
1. Habilitar traza de instrucciones: `--trace` o volcado del PC final.
2. Confirmar que la **última instrucción ejecutada** es:
   - `syscall` con `$v0 = 10` , o
   - instrucción `halt` / `break` equivalente según la ISA implementada.
3. Confirmar que **no** hay instrucciones ejecutadas después del `syscall 10` (el emulador debe detenerse).
4. Confirmar `exit code = 0` del emulador.
5. Confirmar que el emulador **no** entra en estado de pánico, timeout ni excepción no manejada.

Criterio PASS: se cumplen los 5 puntos.

7. Verificación de Número de Ciclos

1. Ejecutar con flag de conteo: `--trace-cycles` o `--stats` .
2. Registrar ciclos totales.
3. Comparar contra el rango esperado de la tabla (sección 5).
4. **Criterio PASS:** `ciclos_min ≤ ciclos_observados ≤ ciclos_max` .
5. **Señales de fallo:**
   - Ciclos = 0 → el programa no se ejecutó.
   - Ciclos > 10× el máximo → posible bucle infinito o mala implementación de branch.
   - Ciclos muy por debajo del mínimo → posible salto incorrecto o terminación prematura

Figura 27. Prompt 4, captura 3 de 4: caso TS-05 (perceptrón), verificación de halt y ciclos.



7. Verificación de Número de Ciclos

1. Ejecutar con flag de conteo: `--trace-cycles` o `--stats`.
2. Registrar ciclos totales.
3. Comparar contra el rango esperado de la tabla (sección 5).
4. **Criterio PASS:** `ciclos_min ≤ ciclos_observados ≤ ciclos_max`.
5. **Señales de fallo:**
   - Ciclos = 0 → el programa no se ejecutó.
   - Ciclos > 10× el máximo → posible bucle infinito o mala implementación de branch.
   - Ciclos muy por debajo del mínimo → posible salto incorrecto o terminación prematura.
6. Ejecutar 3 veces y verificar que el conteo es **idéntico** (determinismo).

8. Criterios de Aceptación Globales

ID	Criterio	Umbral
CA-1	Los 5 programas producen salida exacta esperada	5/5 PASS
CA-2	Los 5 programas terminan con `syscall 10` / `halt`	5/5 PASS
CA-3	Los 5 programas tienen ciclos dentro de rango	5/5 PASS
CA-4	Determinismo (3 ejecuciones idénticas)	5/5 PASS
CA-5	Sin timeouts ni excepciones del emulador	0 fallos

El plan se considera exitoso si CA-1 a CA-5 se cumplen al 100 %.

9. Plantilla de Registro de Resultados

Caso	Salida obtenida	Salida esperada	Match	Última instr.	Halt OK	Ciclos	Rango	Ciclos OK	Estado
TS-01		55					150–600		
TS-02		120					120–500		
TS-03		0.33335					1500–8000		
TS-04		30					200–800		
TS-05		pesos/épocas					2000–20000		

10. Riesgos y Mitigaciones

Riesgo	Mitigación
Entradas hardcodeadas distintas a las asumidas	Revisar el `.asm` y ajustar tabla de valores esperados antes de ejecutar
Formato de salida distinto (decimal vs entero escalado)	Normalizar comparación con tolerancia definida por programa
Conteo de ciclos dependiente de la implementación (pipeline vs monociclo)	Recalibrar rangos tras una ejecución de referencia válida
No determinismo por FP	Usar tolerancia `*1e-4` y comparar con `isclose`

11. Entregables

1. `resultados.csv` con las 5 filas completas.
2. Log de traza (`trace.log`) por programa evidenciando el `syscall 10` final.

Figura 28. Prompt 4, captura 4 de 4: criterios globales, registro de resultados y entregables.

3.5 Prompt 5 — Miguel Damián (código de factorial; prompt literal de la práctica)

Prompt

«A continuación te paso una lista de instrucciones del procesador MIPS que puedo compilar en mi emulador del procesador MIPS. Genera un código en lenguaje ensamblador para el procesador MIPS que implemente el factorial de un número. Incluye la explicación de cada línea de código. Sólo utiliza las instrucciones que te muestro a continuación. `<add, addu, sub, subu, and, or, xor, nor, slt, sltu, sll, srl, mult, div, mflo, mfhi, jr, syscall, addi, addiu, andi, ori, xori, slti, sltiu, lui, lw, sw, beq, bne, j, jal>`»

Resultado: `src/programas/Factorial.java` (25 instrucciones) y su explicación en `docs/programas/factorial_explicacion.md`; al ejecutarlo el emulador muestra “Factorial de 7 = 5040” (63 ciclos).

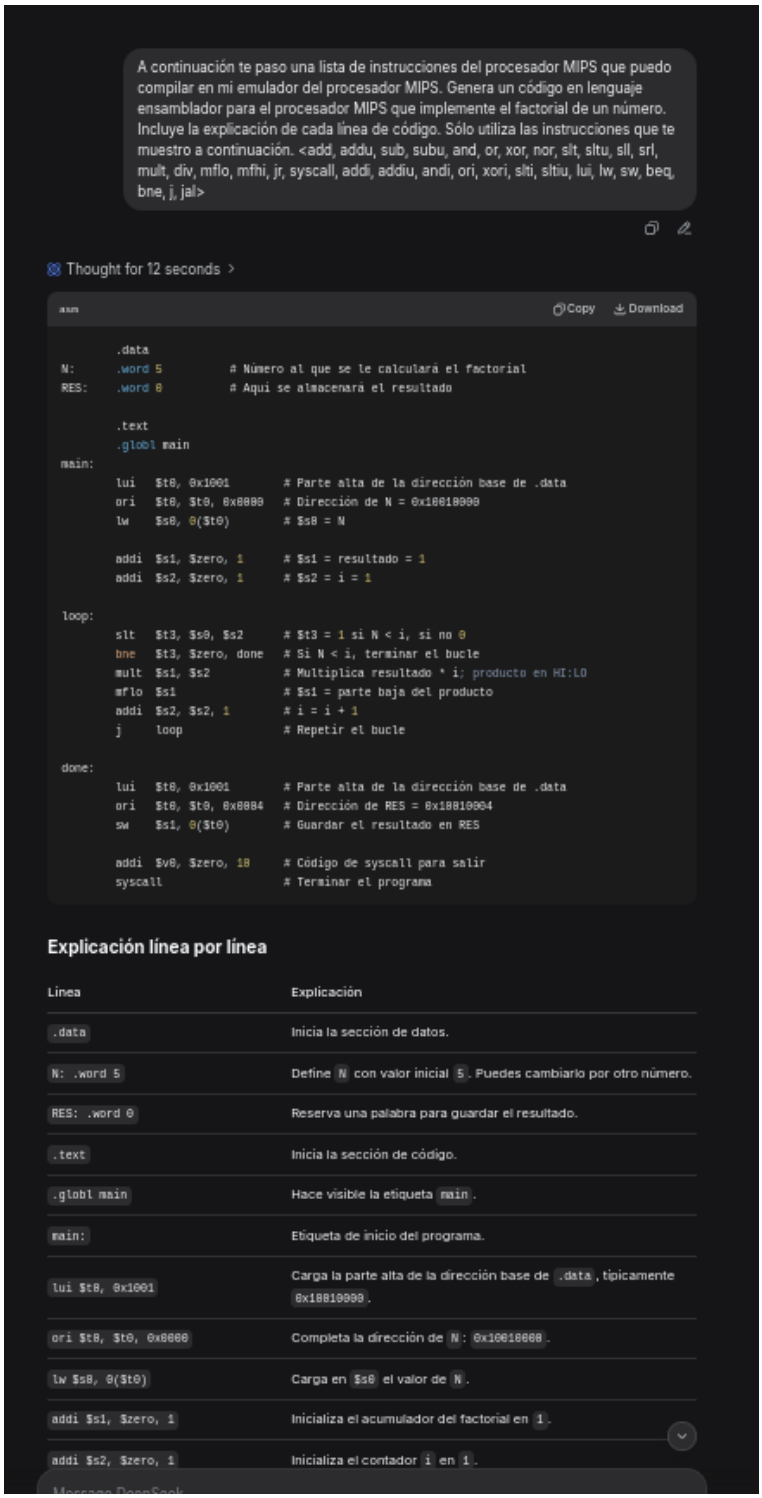


Figura 29. Prompt 5: programa de factorial en ensamblador MIPS y su explicación línea por línea.

3.6 Prompt 6 — Yadid Soto (sumatoria)

Prompt

«Genera un código en lenguaje ensamblador MIPS que realice la sumatoria de los números desde el 3 hasta el 10 usando un bucle con beq/bne y slt para el control. Sólo utiliza las instrucciones de mi emulador: addu, addiu, slt, beq, bne, j, lw, sw y syscall. Incluye la explicación de cada línea.»

Resultado: `src/programas/Sumatoria.java`; el emulador imprime "Sumatoria de 3 a 10 = 52" (68 ciclos).

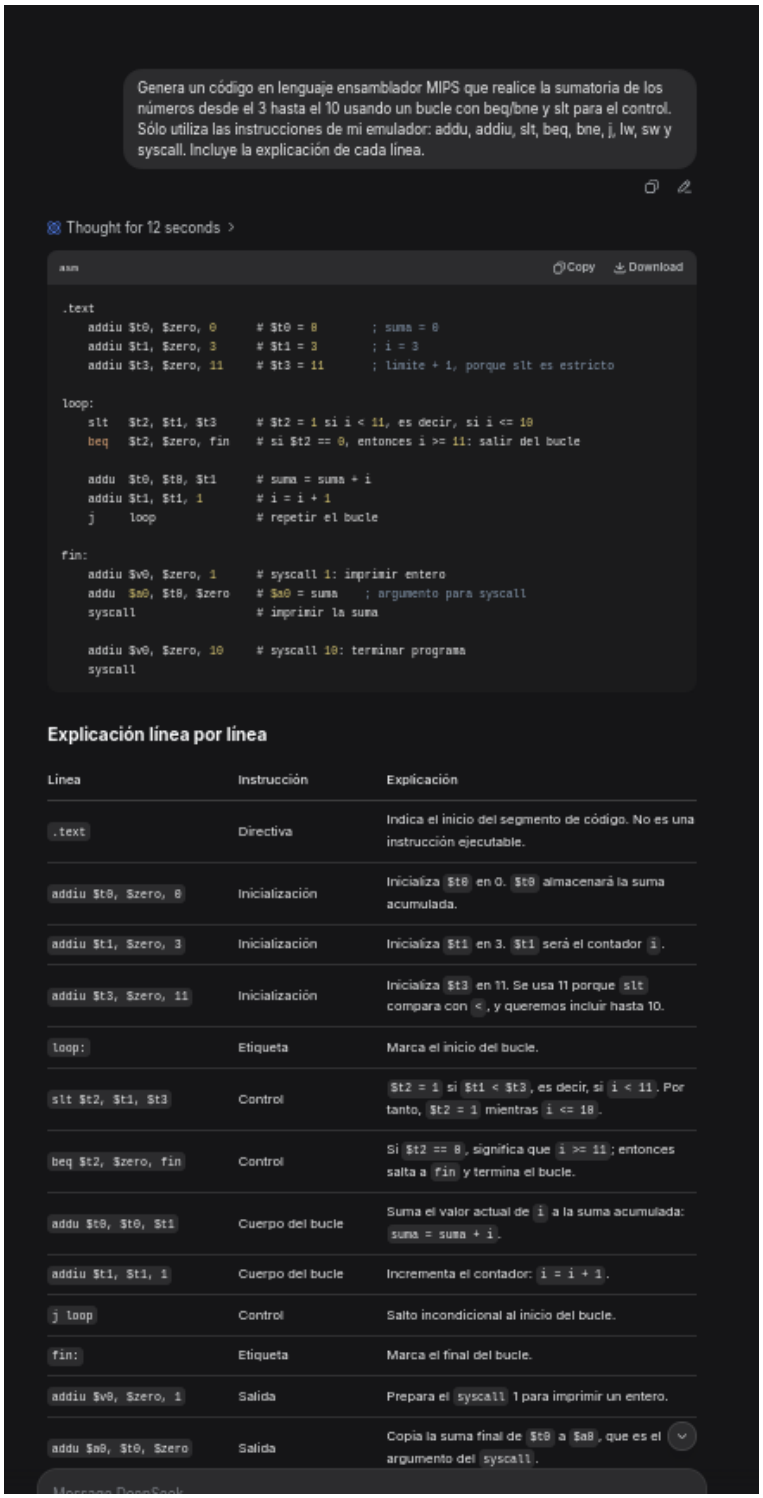


Figura 30. Prompt 6: sumatoria con bucle y control de flujo, explicada línea por línea.

3.7 Prompt 7 — Miguel Damián (integral definida)

Prompt

«Genera código ensamblador MIPS que calcule la integral definida de f(x) = x^2 desde x = 1 hasta x = 5, aproximada con rectángulos de ancho 1 ($1^2 + 2^2 + 3^2 + 4^2$). Usa mult y mflo para elevar al cuadrado, slt/beq para el bucle y syscall para imprimir el resultado con un mensaje. Sólo instrucciones de mi emulador y explica cada línea.»

Resultado: `src/programas/Integral.java`; el emulador imprime “Integral de x^2 dx desde x=1 hasta x=5 = 30” (56 ciclos).



Figura 31. Prompt 7: integral de x^2 con mult/mflo y bucle controlado con slt/beq.

3.8 Prompt 8 — Yadid Soto (promedio de un vector)

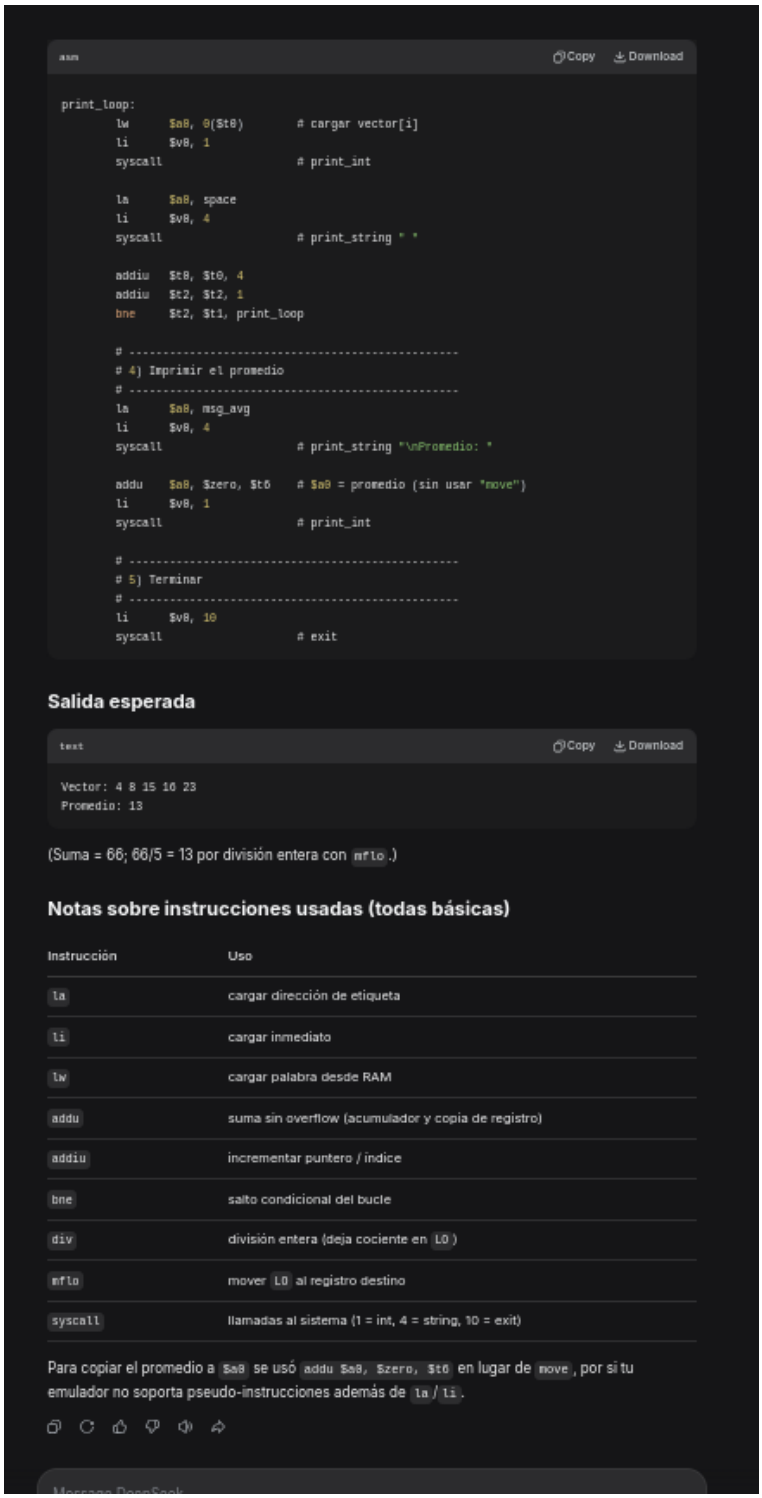
Prompt

«Genera código ensamblador MIPS que calcule el promedio del vector fijo {4, 8, 15, 16, 23} almacenado en RAM: acumula los elementos en un bucle con lw y addu, después divide entre 5 con div y mflo, e imprime el vector y el promedio con los syscalls de cadena y entero. Sólo instrucciones soportadas por mi emulador.»

Resultado: `src/programas/Promedio.java`; el emulador imprime “Vector: 4 8 15 16 23” y “Promedio = 13” (123 ciclos).



Figura 32. Prompt 8, captura 1 de 2: acumulación del vector y cálculo del promedio.



```asm
print_loop:
        lw      $a0, 0($t0)       # cargar vector[i]
        li      $v0, 1
        syscall                   # print_int

        la      $a0, space
        li      $v0, 4
        syscall                   # print_string " "

        addiu   $t0, $t0, 4
        addiu   $t2, $t2, 1
        bne     $t2, $t1, print_loop

        # -----------------------------------------------
        # 4) Imprimir el promedio
        # -----------------------------------------------
        la      $a0, msg_avg
        li      $v0, 4
        syscall                   # print_string "\nPromedio: "

        addu    $a0, $zero, $t6   # $a0 = promedio (sin usar "move")
        li      $v0, 1
        syscall                   # print_int

        # -----------------------------------------------
        # 5) Terminar
        # -----------------------------------------------
        li      $v0, 10
        syscall                   # exit
```

Salida esperada

```text
Vector: 4 8 15 16 23
Promedio: 13
```

(Suma = 66; 66/5 = 13 por división entera con `mflo`.)

Notas sobre instrucciones usadas (todas básicas)

Instrucción	Uso
`la`	cargar dirección de etiqueta
`li`	cargar inmediato
`lw`	cargar palabra desde RAM
`addu`	suma sin overflow (acumulador y copia de registro)
`addiu`	incrementar puntero / índice
`bne`	salto condicional del bucle
`div`	división entera (deja cociente en `LO`)
`mflo`	mover `LO` al registro destino
`syscall`	llamadas al sistema (1 = int, 4 = string, 10 = exit)

Para copiar el promedio a `$a0` se usó `addu $a0, $zero, $t6` en lugar de `move`, por si tu emulador no soporta pseudo-instrucciones además de `la` / `li`.

Figura 33. Prompt 8, captura 2 de 2: impresión del vector y del promedio; notas de instrucciones usadas.

3.9 Prompt 9 — Miguel Damián (perceptrón)

Prompt

«Genera código ensamblador MIPS que evalúe un perceptrón de 3 entradas con datos fijos: entradas x = {1, 1, 0}, pesos w = {2, 2, -3} y bias -1. Acumula el producto punto con mult/mflo en un bucle, suma el bias, aplica el umbral con slt contra $zero y muestra el valor de z y la activación y. Sólo instrucciones de mi emulador, con la explicación de cada línea.»

Resultado: `src/programas/Perceptron.java`; el emulador imprime "z = 3 y = 1" (62 ciclos; $z = 2 \cdot 1 + 2 \cdot 1 + (-3) \cdot 0 - 1 = 3$, y = 1 porque $z \geq 0$).



Figura 34. Prompt 9: perceptrón de 3 entradas con producto punto, bias y umbral.

3.10 Prompt 10 — Yadid Soto (clase ALU)

Prompt

«Implementa en Java la clase ALU de un procesador MIPS de 32 bits con las operaciones ADD, ADDU, SUB, SUBU, AND, OR, XOR, NOR, SLT, SLTU, SLL, SRL y LUI, más NOP. Detecta el desbordamiento con signo en ADD y SUB y detiene el programa en ese caso, mientras que ADDU y SUBU envuelven el resultado. Devuelve también el resultado de las comparaciones SLT y SLTU.»

Resultado: `src/alu/ALU.java` y `src/alu/OperacionALU.java` (13 operaciones + NOP), probadas con las pruebas unitarias de la ALU.



Figura 35. Prompt 10, captura 1 de 2: implementación de la ALU de 32 bits en Java.



```java
    /** Sobrecarga de conveniencia para operaciones sin shamt. */
    public Result execute(Operation op, int rs, int rt) {
        return execute(op, rs, rt, 0);
    }

    /* ================== Pruebas ================== */
    public static void main(String[] args) {
        ALU alu = new ALU();

        System.out.println("ADD 5+3        -> " + alu.execute(Operation.ADD, 5, 3));
        System.out.println("ADDU MAX+1     -> " + alu.execute(Operation.ADDU, Integer.MAX_VA
LUE, 1));
        System.out.println("SUBU MIN-1     -> " + alu.execute(Operation.SUBU, Integer.MIN_VA
LUE, 1));
        System.out.println("AND  0xF0,0x3C -> " + alu.execute(Operation.AND, 0xF0, 0x3C));
        System.out.println("NOR  0,0       -> " + alu.execute(Operation.NOR, 0, 0));
        System.out.println("SLT  -1,1      -> " + alu.execute(Operation.SLT, -1, 1));
        System.out.println("SLTU -1,1      -> " + alu.execute(Operation.SLTU, -1, 1));
        System.out.println("SLL  1<<4      -> " + alu.execute(Operation.SLL, 1, 0, 4));
        System.out.println("SRL  -1>>>28   -> " + alu.execute(Operation.SRL, -1, 0, 28));
        System.out.println("LUI  0x1234    -> " + alu.execute(Operation.LUI, 0, 0x1234));

        try {
            alu.execute(Operation.ADD, Integer.MAX_VALUE, 1);
        } catch (OverflowException e) {
            System.out.println("HALT (ADD): " + e.getMessage());
        }
        try {
            alu.execute(Operation.SUB, Integer.MIN_VALUE, 1);
        } catch (OverflowException e) {
            System.out.println("HALT (SUB): " + e.getMessage());
        }
    }
}
```

Puntos de diseño clave

1. **Representación de 32 bits con `int`**: en Java `int` es exactamente 32 bits con signo, así que las operaciones bit a bit (AND, OR, XOR, NOR) y los desplazamientos funcionan tal cual.
2. **Detección de desbordamiento con signo** (sólo `ADD` y `SUB`):
   - `ADD`: hay desbordamiento si `rs` y `rt` tienen el mismo signo pero el resultado tiene signo distinto → `((rs ^ r) & (rt ^ r)) < 0`.
   - `SUB`: hay desbordamiento si `rs` y `rt` tienen signos distintos y el resultado tiene signo distinto de `rs` → `((rs ^ rt) & (rs ^ r)) < 0`.
   - Al detectarlo se lanza `OverflowException`; el simulador la captura y **detiene el programa**.
3. `ADDU` / `SUBU`: no comprueban overflow; Java ya envuelve de forma natural en aritmética de 32 bits (`MAX_VALUE + 1 → MIN_VALUE`).
4. `SLT` vs `SLTU`:
   - `SLT` usa `<` sobre `int` (comparación con signo).
   - `SLTU` usa `Integer.compareUnsigned` (comparación sin signo).
   - Ambos devuelven **además** un `boolean comparison` con el resultado de la comparación, tal como se pidió, independientemente del valor 1/0.
5. `SLL` / `SRL`: máscara `shamt & 0x1F` para que solo se usen los 5 bits bajos (rango válido 0–31). `SRL` usa `>>>` (desplazamiento lógico, rellena con ceros).
6. `LUI`: `rt << 16` coloca el inmediato de 16 bits en la mitad alta del registro destino.
7. `NOP`: devuelve 0 sin efectos.
8. **Sobrecarga** `execute(op, rs, rt)`: azúcar sintáctico para no tener que pasar `shamt` en operaciones que no lo usan.

Figura 36. Prompt 10, captura 2 de 2: pruebas de la ALU y puntos de diseño clave.

3.11 Prompt 11 — Miguel Damián (ventana gráfica del emulador)

Prompt

«Genera una ventana en Java (Swing) para mi emulador del procesador MIPS con estos campos: instrucciones en binario (32 bits por línea, acepta hexadecimal y comentarios con #), historial con Ciclo, PC, Instrucción, Operación, señales de control y PC siguiente, los 32 registros, la memoria por segmentos y el estado CPU (PC, HI, LO); botones: Ejecutar 1 ciclo, Ejecutar programa, Cargar en el PC, Cargar ejemplo, Reiniciar CPU y Limpiar historial.»

Resultado: `src/ui/VentanaEmulador.java`, usada para observar el estado de la CPU ciclo a ciclo y las señales de control.



Figura 37. Prompt 11, captura 1 de 2: ventana Swing con historial, registros y estado de la CPU.



```java
    private void refreshRegisters() {
        for (int i = 0; i < 32; i++) {
            regModel.setValueAt(String.format("0x%08X", regs[i]), i, 2);
            regModel.setValueAt(String.valueOf(regs[i]), i, 3);
        }
        regTable.repaint();
    }

    private void refreshStatus() {
        lblPc.setText(String.format("PC: 0x%08X", pc));
        lblHi.setText(String.format("HI: 0x%08X", hi));
        lblLo.setText(String.format("LO: 0x%08X", lo));
        lblCiclo.setText("Ciclo: " + cycle);
    }

    private void refreshMemory() {
        StringBuilder sb = new StringBuilder();
        sb.append("=== SEGMENTO DE TEXTO (0x00400000) ===\n");
        if (instrMem.isEmpty()) {
            sb.append("(sin instrucciones cargadas)\n");
        } else {
            for (int i = 0; i < instrMem.size(); i++) {
                int addr = INSTR_BASE + i * 4;
                int w = instrMem.get(i);
                sb.append(String.format("0x%08X  %08X  %s%n", addr, w, disasm(w, addr)));
            }
        }
        sb.append("\n=== SEGMENTO DE DATOS / PILA (palabras no nulas) ===\n");
        if (memory.isEmpty()) {
            sb.append("(vacio)\n");
        } else {
            for (Map.Entry<Integer,Integer> e : memory.entrySet()) {
                sb.append(String.format("0x%08X  %08X  %d%n",
                        e.getKey(), e.getValue(), e.getValue()));
            }
        }
        memoryArea.setText(sb.toString());
        memoryArea.setCaretPosition(0);
    }

    private void setStatus(String s) {
        if (lblEstado != null) lblEstado.setText(s);
    }
}
```

Notas de uso

- **Formato de entrada**: cada línea es una instrucción de 32 bits. Se acepta binario (`00000001010011010100000000100000`) o hexadecimal (`0x01094020` o `01094020`). Todo lo que esté después de `#` se ignora.
- `Cargar en el PC` : ensambla el editor y posiciona el PC en `0x00400000` . Si hay errores, se listan por número de línea.
- `Ejecutar 1 ciclo` : ejecuta una instrucción, agrega una fila al historial y refresca registros/memoria/estado.
- `Ejecutar programa` : itera hasta fin de instrucciones o detección de bucle (límite de 50 000 ciclos).
- `Cargar ejemplo` : carga el programa `1+2+3+4+5` ; al ejecutarlo, `$t0 = 15` y `mem[0] = 15` .
- **Subconjunto MIPS soportado**: `add/addu/sub/subu/and/or/xor/nor/slt/sll/srl/sra/jr/mfhi/mthi/mflo/mtlo/mult/multu` , `addi/addiu/slti/andi/ori/xori/lui/lw/sw/beq/bne` , `j/jal` y `nop` . Las señales de control se muestran de forma didáctica (`RegWr` , `RegDst` , `ALUSrc` , `MemRead` , `MemWrite` , `Branch` , `ALUOp`).
- La memoria es dispersa (se guardan solo las palabras escritas con `sw`), lo que mantiene la vista de memoria legible.

Figura 38. Prompt 11, captura 2 de 2: refresco de registros y memoria, y notas de uso.

3.12 Prompt 12 — José de Jesús Berman (documentación del conjunto de instrucciones)

Prompt

«Genera la documentación del conjunto de instrucciones del procesador MIPS que soporta mi emulador: formato R con su campo funct, formato I con su opcode, formato J, los efectos de cada instrucción sobre los registros en binario de 32 bits y la tabla de servicios de syscall con sus argumentos.»

Resultado: `docs/instrucciones.md`, referencia para escribir y revisar los cinco programas de usuario.



Figura 39. Prompt 12, captura 1 de 4: formatos de instrucción MIPS32 e instrucciones de formato R.



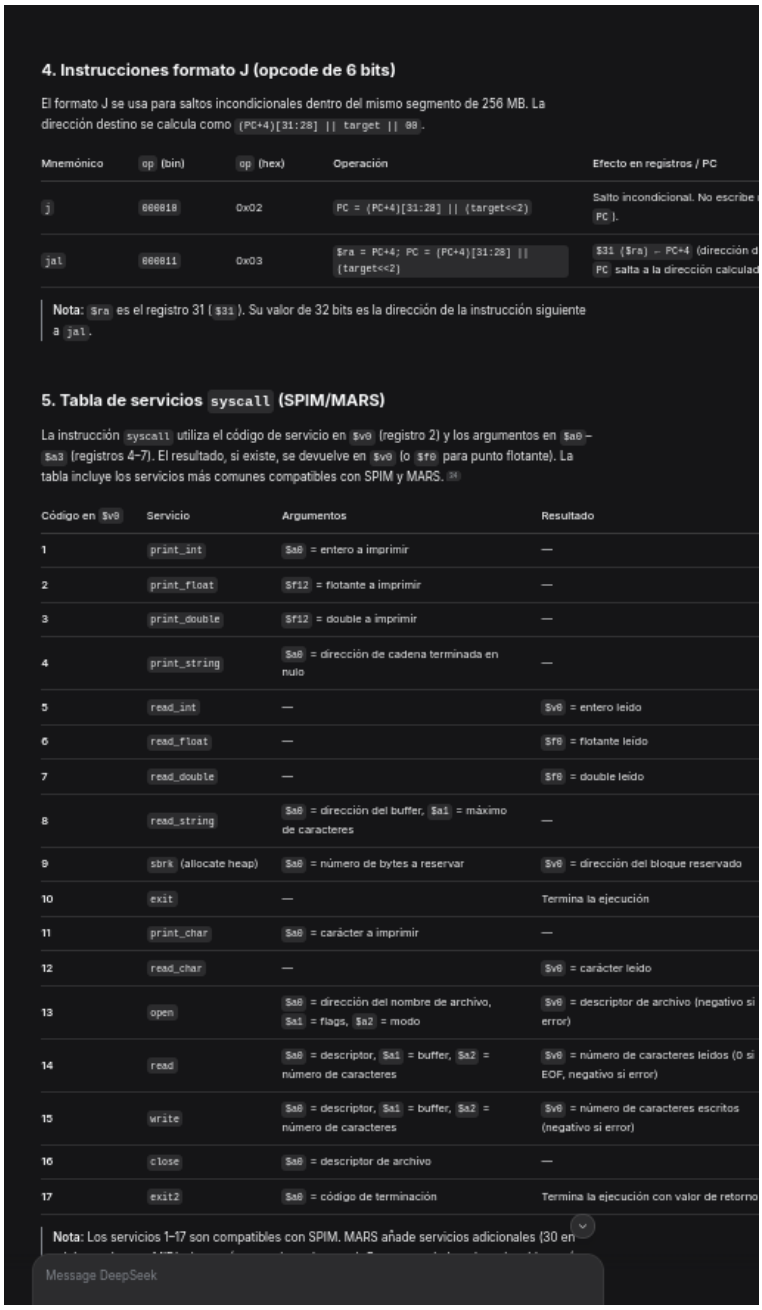
3. Instrucciones formato I (opcode de 6 bits)

El campo `op` (6 bits) identifica la instrucción. El inmediato (`imm`) de 16 bits se extiende con signo (para aritmética y branches) o con cero (para lógicas y `lui`). Los branches comparan registros de 32 bits y actualizan el `PC` si la condición se cumple.

Mnemónico	`op` (bin)	`op` (hex)	Operación	Efecto en registros / PC (32 bits
`addi`	`001000`	0x08	`rt = rs + imm` (con signo)	`rt ← {rs + SignExt(imm)}` (32 overflow) [30]
`addiu`	`001001`	0x09	`rt = rs + imm` (sin signo)	`rt ← {rs + SignExt(imm)}` (32 trap) [30]
`andi`	`001100`	0x0C	`rt = rs & imm` (cero-extendido)	`rt ← {rs AND ZeroExt(imm)}` (
`ori`	`001101`	0x0D	`rt = rs | imm` (cero-extendido)	`rt ← {rs OR ZeroExt(imm)}` (
`xori`	`001110`	0x0E	`rt = rs ^ imm` (cero-extendido)	`rt ← {rs XOR ZeroExt(imm)}` (
`slti`	`001010`	0x0A	`rt = (rs < imm) ? 1 : 0` (con signo)	`rt ← 0x00000001` si `rs < Sign` no `0x00000000` [30]
`sltiu`	`001011`	0x0B	`rt = (rs < imm) ? 1 : 0` (sin signo)	`rt ← 0x00000001` si `rs < Zero` no `0x00000000` [30]
`lui`	`001111`	0x0F	`rt = imm << 16`	`rt ← {imm << 16}` (carga inme parte alta) [30]
`lw`	`100011`	0x23	`rt = MEM[rs + imm]`	'rt ← palabra de 32 bits cargad memoria | [30]
`sw`	`101011`	0x2B	`MEM[rs + imm] = rt`	Almacena `rt` (32 bits) en mem modifica registros (excepto mer
`lb`	`100000`	0x20	`rt = SignExt(MEM[rs+imm][7:0])`	`rt ← SignExt(byte cargado)` (
`lbu`	`100100`	0x24	`rt = ZeroExt(MEM[rs+imm][7:0])`	`rt ← ZeroExt(byte cargado)` (
`lh`	`100001`	0x21	`rt = SignExt(MEM[rs+imm][15:0])`	`rt ← SignExt(media palabra c` bits) [30]
`lhu`	`100101`	0x25	`rt = ZeroExt(MEM[rs+imm][15:0])`	`rt ← ZeroExt(media palabra c` bits) [30]
`sb`	`101000`	0x28	`MEM[rs+imm] = rt[7:0]`	Almacena el byte bajo de `rt` . N registros. [30]
`sh`	`101001`	0x29	`MEM[rs+imm] = rt[15:0]`	Almacena la media palabra baja modifica registros. [30]
`beq`	`000100`	0x04	`if (rs == rt) PC = PC+4 + (imm<<2)`	Si `rs == rt` , salta. No escribe PC). [30]
`bne`	`000101`	0x05	`if (rs != rt) PC = PC+4 + (imm<<2)`	Si `rs != rt` , salta. No escribe PC). [30]
`blez`	`000110`	0x06	`if (rs <= 0) PC = PC+4 + (imm<<2)`	Si `rs ≤ 0` (con signo), salta. N registro. [30]
`bgtz`	`000111`	0x07	`if (rs > 0) PC = PC+4 + (imm<<2)`	Si `rs > 0` (con signo), salta. N registro. [30]
`bltz`	`000001`	0x01	`if (rs < 0) PC = PC+4 + (imm<<2)` (rt= `00000`)	Si `rs < 0` , salta. No escribe re
`bgez`	`000001`	0x01	`if (rs >= 0) PC = PC+4 + (imm<<2)` (rt= `00001`)	Si `rs ≥ 0` , salta. No escribe re

Message DeepSeek

Figura 40. Prompt 12, captura 2 de 4: instrucciones de formato I.



4. Instrucciones formato J (opcode de 6 bits)

El formato J se usa para saltos incondicionales dentro del mismo segmento de 256 MB. La dirección destino se calcula como `(PC+4)[31:28] || target || 00`.

Mnemónico	`op` (bin)	`op` (hex)	Operación	Efecto en registros / PC
`j`	`000010`	0x02	`PC = (PC+4)[31:28] || (target<<2)`	Salto incondicional. No escribe `PC`).
`jal`	`000011`	0x03	`$ra = PC+4; PC = (PC+4)[31:28] || (target<<2)`	`$31` (`$ra`) ← `PC+4` (dirección d `PC` salta a la dirección calculad

> **Nota:** `$ra` es el registro 31 (`$31`). Su valor de 32 bits es la dirección de la instrucción siguiente a `jal`.

5. Tabla de servicios `syscall` (SPIM/MARS)

La instrucción `syscall` utiliza el código de servicio en `$v0` (registro 2) y los argumentos en `$a0`–`$a3` (registros 4–7). El resultado, si existe, se devuelve en `$v0` (o `$f0` para punto flotante). La tabla incluye los servicios más comunes compatibles con SPIM y MARS.

Código en `$v0`	Servicio	Argumentos	Resultado
1	`print_int`	`$a0` = entero a imprimir	—
2	`print_float`	`$f12` = flotante a imprimir	—
3	`print_double`	`$f12` = double a imprimir	—
4	`print_string`	`$a0` = dirección de cadena terminada en nulo	—
5	`read_int`	—	`$v0` = entero leído
6	`read_float`	—	`$f0` = flotante leído
7	`read_double`	—	`$f0` = double leído
8	`read_string`	`$a0` = dirección del buffer, `$a1` = máximo de caracteres	—
9	`sbrk` (allocate heap)	`$a0` = número de bytes a reservar	`$v0` = dirección del bloque reservado
10	`exit`	—	Termina la ejecución
11	`print_char`	`$a0` = carácter a imprimir	—
12	`read_char`	—	`$v0` = carácter leído
13	`open`	`$a0` = dirección del nombre de archivo, `$a1` = flags, `$a2` = modo	`$v0` = descriptor de archivo (negativo si error)
14	`read`	`$a0` = descriptor, `$a1` = buffer, `$a2` = número de caracteres	`$v0` = número de caracteres leídos (0 si EOF, negativo si error)
15	`write`	`$a0` = descriptor, `$a1` = buffer, `$a2` = número de caracteres	`$v0` = número de caracteres escritos (negativo si error)
16	`close`	`$a0` = descriptor de archivo	—
17	`exit2`	`$a0` = código de terminación	Termina la ejecución con valor de retorno

> **Nota:** Los servicios 1–17 son compatibles con SPIM. MARS añade servicios adicionales (30 en

Message DeepSeek

Figura 41. Prompt 12, captura 3 de 4: instrucciones de formato J y tabla de servicios syscall.



Código en $v0	Servicio	Argumentos	Resultado
1	print_int	$a0 = entero a imprimir	—
2	print_float	$f12 = flotante a imprimir	—
3	print_double	$f12 = double a imprimir	—
4	print_string	$a0 = dirección de cadena terminada en nulo	—
5	read_int	—	$v0 = entero leído
6	read_float	—	$f0 = flotante leído
7	read_double	—	$f0 = double leído
8	read_string	$a0 = dirección del buffer, $a1 = máximo de caracteres	—
9	sbrk (allocate heap)	$a0 = número de bytes a reservar	$v0 = dirección del bloque reservado
10	exit	—	Termina la ejecución
11	print_char	$a0 = carácter a imprimir	—
12	read_char	—	$v0 = carácter leído
13	open	$a0 = dirección del nombre de archivo, $a1 = flags, $a2 = modo	$v0 = descriptor de archivo (negativo si error)
14	read	$a0 = descriptor, $a1 = buffer, $a2 = número de caracteres	$v0 = número de caracteres leídos (0 si EOF, negativo si error)
15	write	$a0 = descriptor, $a1 = buffer, $a2 = número de caracteres	$v0 = número de caracteres escritos (negativo si error)
16	close	$a0 = descriptor de archivo	—
17	exit2	$a0 = código de terminación	Termina la ejecución con valor de retorno

> **Nota:** Los servicios 1–17 son compatibles con SPIM. MARS añade servicios adicionales (30 en adelante: tiempo, MIDI, sleep, números aleatorios, etc.). Para tu emulador, ajusta la tabla según los servicios que hayas implementado.

6. Resumen de codificación binaria (32 bits)

Para construir una instrucción en binario, concatena los campos en el orden indicado:

- **R:** `[op=000000] [rs] [rt] [rd] [shamt] [funct]`
- **I:** `[op] [rs] [rt] [imm]`
- **J:** `[op] [target]`

Cada registro se codifica con 5 bits (0–31). El registro `$zero` es `00000` y siempre vale `0x00000000`. Los efectos en los registros siempre son de 32 bits, y los inmediatos se extienden a 32 bits antes de operar (con signo o cero según la instrucción).

Esta documentación cubre el núcleo del conjunto de instrucciones MIPS32 que tu emulador debe soportar. Si tu emulador incluye instrucciones adicionales (p. ej., `mul`, `rem`, `movn`, `movz`, o extensiones), se recomienda añadirlas a las tablas correspondientes siguiendo el mismo formato.

60 web pages

Message DeepSeek

Figura 42. Prompt 12, captura 4 de 4: tabla de servicios syscall y resumen de codificación binaria.

3.13 Prompt 13 — Equipo (reestructuración final a cinco paquetes)

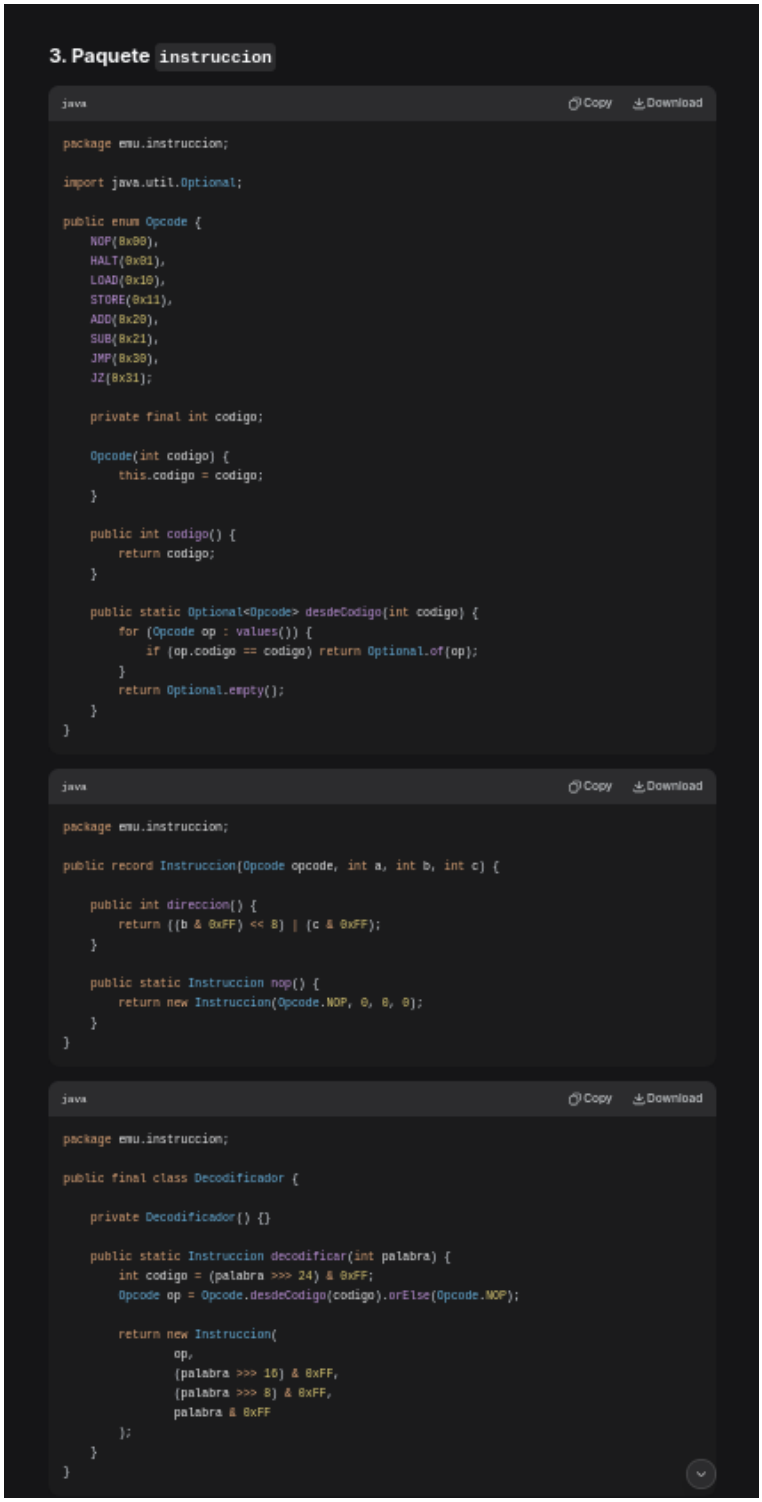
Prompt

«Reorganiza el emulador en 5 paquetes grandes (cpu, ejecucion, memoria, alu, instruccion) al estilo del diagrama del compañero, con la memoria segmentada por dirección (ROM texto@0, RAM datos@100, disco@200), una CPU que nunca muere (arranca en 0, PC circular, memoria en 0 = NOP, límite de 100000 ciclos) y una instrucción HALT para que la CPU sepa que el programa terminó; añade pruebas y regenera los diagramas.»

Resultado: la estructura actual del proyecto, los diagramas de clases en `docs/diagramas/` y las 201 pruebas (117 + 44 + 40) que pasan con `./probar.sh`.



Figura 43. Prompt 13, captura 1 de 4: árbol de paquetes propuesto, mapa de memoria y paquete memoria.



3. Paquete `instruccion`

java

Copy Download

```java
package emu.instruccion;

import java.util.Optional;

public enum Opcode {
    NOP(0x00),
    HALT(0x01),
    LOAD(0x10),
    STORE(0x11),
    ADD(0x20),
    SUB(0x21),
    JMP(0x30),
    JZ(0x31);

    private final int codigo;

    Opcode(int codigo) {
        this.codigo = codigo;
    }

    public int codigo() {
        return codigo;
    }

    public static Optional<Opcode> desdeCodigo(int codigo) {
        for (Opcode op : values()) {
            if (op.codigo == codigo) return Optional.of(op);
        }
        return Optional.empty();
    }
}
```

java

Copy Download

```java
package emu.instruccion;

public record Instruccion(Opcode opcode, int a, int b, int c) {

    public int direccion() {
        return ((b & 0xFF) << 8) | (c & 0xFF);
    }

    public static Instruccion nop() {
        return new Instruccion(Opcode.NOP, 0, 0, 0);
    }
}
```

java

Copy Download

```java
package emu.instruccion;

public final class Decodificador {

    private Decodificador() {}

    public static Instruccion decodificar(int palabra) {
        int codigo = (palabra >>> 24) & 0xFF;
        Opcode op = Opcode.desdeCodigo(codigo).orElse(Opcode.NOP);

        return new Instruccion(
                op,
                (palabra >>> 16) & 0xFF,
                (palabra >>> 8) & 0xFF,
                palabra & 0xFF
        );
    }
}
```

Figura 44. Prompt 13, captura 2 de 4: paquete instrucción (Opcode, Instrucción y Decodificador).



6. Paquete `cpu`

java — Copy — Download

```java
package emu.cpu;

public record RunResult(Motivo motivo, long ciclos, int pc) {

    public enum Motivo {
        HALT,
        LIMITE_CICLOS
    }
}
```

java — Copy — Download

```java
package emu.cpu;

import emu.alu.Alu;
import emu.ejecucion.ContextoEjecucion;
import emu.ejecucion.Ejecutor;
import emu.memoria.Memoria;

public final class CPU {

    public static final int MAX_CICLOS_DEFAULT = 100_000;

    private final ContextoEjecucion ctx;
    private final Ejecutor ejecutor;
    private final int maxCiclos;

    public CPU(Memoria memoria) {
        this(memoria, MAX_CICLOS_DEFAULT);
    }

    public CPU(Memoria memoria, int maxCiclos) {
        if (maxCiclos <= 0) {
            throw new IllegalArgumentException("El límite de ciclos debe ser positivo");
        }
        this.ctx = new ContextoEjecucion(memoria);
        this.ejecutor = new Ejecutor(new Alu());
        this.maxCiclos = maxCiclos;
    }

    public RunResult ejecutar() {
        while (!ctx.isHalt() && ctx.getCiclos() < maxCiclos) {
            int palabra = ctx.fetch();
            ejecutor.ejecutar(ctx, palabra);
            ctx.incrementarCiclo();
        }

        if (ctx.isHalt()) {
            return new RunResult(RunResult.Motivo.HALT, ctx.getCiclos(), ctx.getPc());
        }

        return new RunResult(RunResult.Motivo.LIMITE_CICLOS, ctx.getCiclos(), ctx.getPc());
    }

    public void reiniciar() {
        ctx.reset(0);
    }
}
```

Comportamiento clave:

- Arranca en `PC = 0`.
- `PC` es circular: `Math.floorMod(..., Memoria.TAM)`.
- Memoria no inicializada `0` es `NOP`.
- `HALT` marca fin de programa y `ejecutar()` devuelve `Motivo.HALT`.
- Si no hay `HALT`, al llegar a `100000` ciclos devuelve `Motivo.LIMITE_CICLOS`.

Figura 45. Prompt 13, captura 3 de 4: paquete cpu (CPU con límite de ciclos y HALT).



Figura 46. Prompt 13, captura 4 de 4: pruebas JUnit 5 de memoria y cpu.

3.14 Reflexión sobre el uso de la inteligencia artificial

La IA se usó como copiloto: análisis del diseño, planes de pruebas, programas de usuario, ventana y explicaciones. Las decisiones las tomó el equipo: elegir MIPS, la segmentación de la memoria por dirección, el `HALT` propio (opcode `0x3F`), la CPU que nunca muere y el límite de 100 000 ciclos. La validación también: correr las 201 pruebas y las capturas de `salidas/`. Varios resultados iniciales de la IA no compilaban o no se comportaban como se esperaba (por ejemplo, las señales de control, que se reconstruyeron con un Builder para evitar errores de argumentos posicionales) y se corrigieron a mano.

En síntesis, la IA ahorra horas, pero sólo es útil si se sabe qué pedir y se puede comprobar la respuesta con pruebas automatizadas.

4. Reflexiones finales

4.1 Nivel de programación que se requiere

Implementar este tipo de emuladores no exige ser un programador experto, pero sí un nivel intermedio sólido y, sobre todo, orden. Se necesita dominar programación orientada a objetos en Java (clases, paquetes y encapsulamiento; incluso algo de herencia, como `DiscoDuro` que se deriva de `UnidadAlmacenamiento`), manejar arreglos y operaciones a nivel de bits, y estar cómodo con números binarios y hexadecimales.

El verdadero reto no es escribir la primera versión del código, sino depurar: cuando la salida no coincide con la esperada hay que rastrear el estado del PC, los registros y la memoria ciclo por ciclo. También se requiere capacidad de leer una especificación técnica (el manual de MIPS) y traducirla a código, más el hábito de escribir pruebas antes de dar por bueno un módulo. En síntesis, el nivel de programación es el de un estudiante avanzado, pero lo que hace que el proyecto funcione es la constancia y la disciplina de la verificación.

4.2 El trabajo en equipo en el diseño e implementación de un procesador

Un procesador, aun tratándose de un emulador básico, es una tarea demasiado grande para una sola persona, y esta práctica lo confirmó. Repartimos el trabajo por módulos: Yadid trabajó las unidades de almacenamiento y la ALU, además de los programas de sumatoria y promedio; José de Jesús armó el sistema de pruebas (unitarias, de integración y de sistema), la documentación técnica y el DiscoDuro; Miguel implementó los programas de factorial, integral y perceptrón y la ventana gráfica.

Lo más importante no fue el reparto, sino la integración: acordamos desde el inicio conven-

ciones comunes (nombres de paquetes, ancho de palabra, mapa de direcciones y firmas de las clases), nos revisamos el código mutuamente y corríamos las tres suites de pruebas antes de dar por concluida cualquier tarea. Comparar nuestro diagrama de clases con el de un compañero fue especialmente útil: nos obligó a explicitar las decisiones de diseño y a unificar el esqueleto de cinco paquetes antes de escribir el código. Así como en la industria existen equipos separados de diseño, verificación y firmware, nosotros separamos la construcción del emulador de su verificación, y eso nos permitió encontrar errores que un solo programador probablemente no habría visto.

4.3 Por qué les conviene a quienes cursarán esta experiencia el próximo año

A los estudiantes que tomen este curso el próximo año les conviene este conocimiento por tres razones prácticas. La primera es que conecta todas las capas de la computación: quien entiende el ciclo de un procesador entiende qué hace realmente un compilador, por qué una instrucción cuesta lo que cuesta y cómo se comporta la memoria; deja de programar a ciegas. La segunda es que mejora como programador: depurar a nivel de registros enseña a razonar con estado y con tiempos, habilidad aplicable a sistemas embebidos, IoT, sistemas operativos y verificación de hardware, áreas con demanda laboral constante. La tercera es que aprenden a usar la inteligencia artificial con criterio: en esta práctica la IA nos ahorró muchas horas, pero sólo funcionó porque sabíamos qué pedir y podíamos comprobar la respuesta con pruebas; sin esos dos componentes, la herramienta devuelve código que parece correcto y no lo es. A esto se suma la experiencia de trabajar en equipo y entregar un reporte formal, situaciones que se parecen cada vez más a lo que será su vida profesional.

4.4 Valores en juego

La práctica puso en acción los valores de la universidad: libertad para elegir cómo estructurar cada módulo y qué decisiones de diseño tomar; responsabilidad por la parte que a cada quien le tocó y por entregar el trabajo en la fecha y por el medio establecidos; solidaridad al apoyarnos cuando un compañero se atascaba y al compartir lo que entendíamos; respeto por las ideas y el estilo de los demás, sin imponer la propia solución; honestidad al reportar exactamente lo que las pruebas arrojaron, incluidos los errores que encontramos, sin maquillar resultados; e integridad al cumplir con lo que nos comprometimos, presentando nuestro propio trabajo y utilizando la inteligencia artificial únicamente como herramienta de apoyo, dejando constancia de su uso en este reporte.

5. Referencias

- Patterson, D. A., & Hennessy, J. L. (2009). Computer organization and design: The hardware/software interface (4.ª ed.). Morgan Kaufmann.
- Harris, D. G., & Harris, S. L. (2012). Digital design and computer architecture: MIPS Edition. Morgan Kaufmann.
- Hamacher, V. C., Vranesic, Z. G., Zaky, S. A., & Manjikian, N. (2002). Computer organization (5.ª ed.). McGraw-Hill.
- MIPS Technologies. (s. f.). MIPS32 architecture for programmers, volume I: Introduction to the MIPS32 architecture. Recuperado de https://www.mips.com/
- Universidad Veracruzana. (2026). Práctica 2. Emulador básico de un procesador [Instrucciones de la asignatura y plantilla del reporte]. Eminus.